Project Title: LLVM OPTIMISATIONS FOR ENERGY EFFICIENT CODE VIA SOFTWARE PIPELINING

Project Supervisor:	**Contact Details:**
Oswaldo Cadenas	S4300390@lsbu.ac.uk

Project Objectives (3-5 lines)

This project aims to investigate how LLVM compiler optimisation settings affect runtime, power consumption, and estimated energy usage. The project focuses on loop-intensive C benchmarks compiled with different LLVM optimisation levels. The experiments will be carried out on a Raspberry Pi 5 using system-level power readings. The results will be compared to identify whether compiler optimisation improves energy efficiency on low-power hardware

Project Description (2-3 paragraphs)

This project develops an experimental framework to measure the energy impact of LLVM compiler optimisations. Selected PolyBench/C benchmark programs will be compiled using different optimisation settings, including O0, O2, O3, and O3 with vectorisation disabled. The compiled programs will then be executed on a Raspberry Pi 5 while runtime and power data are recorded.

The collected data will be analysed to compare execution time, average power, and estimated energy consumption across the optimisation settings. The project will also inspect generated assembly code to understand how optimisation changes the low-level structure of the benchmark programs. The main purpose is to evaluate whether LLVM optimisation can improve energy efficiency for loop-intensive programs on real low-power hardware.

Project Output

- ☑ System Design and evaluation
- ☑ Comparative overview or study and evaluation framework
- ☑ System Analysis and Modelling
- ☐ Theoretical analysis, Algorithm Design and Development

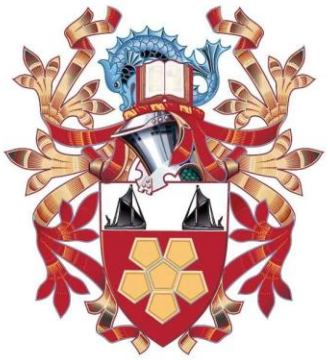
Required HW/SW

WHAT	**WHERE (e.g. University's premises, Student's PC etc.)**
Raspberry Pi 5	Students Own Hardware
Raspberry Pi OS 64-bit	Raspberry Pi 5
LLVM/Clang 18.1.0	Raspberry Pi 5
Python	Raspberry Pi 5 and Google Colab
PolyBench/C benchmark Suite	Raspberry Pi 5

Other Requirements

WHAT	**LEVEL**	**IMPACT**
Programming	Intermediate	Required to write and run build, logging, and controller scripts.
Algorithm Design	Basic	Required only for understanding compiler optimisation behaviour, not for creating a new algorithm.
Experimental Evaluation	Intermediate	Required to collect, process, and compare runtime, power, and energy results.
Data analysis	Intermediate	Required to calculate averages, energy estimates, and produce tables/graphs.

LLVM Optimisations for Energy Efficient Code via Software Pipelining



London South Bank University

EST 1892

School of Engineering

BSc in Computer Science

Student ID: 4300390

Declaration

I, Abdihakim Burale, certify that this work is my own and was completed in compliance with the guidelines of academic integrity. I declare that all sources used in this project have been cited and referenced appropriately to acknowledge the original authors.

I understand that plagiarism, fabrication, collusion, or any other form of academic dishonesty breaches institutional policies and ethical standards. I have taken appropriate steps to ensure that this submission does not contain any such misconduct.

Furthermore, I confirm that no part of this submission has previously been submitted for academic credit at this or any other institution. Any resemblance to existing work is either coincidental or has been properly referenced.

I understand that failure to comply with the institution's academic-integrity policies may result in disciplinary action. By submitting this work, I affirm my commitment to honesty, integrity, and responsible scholarship.

Name: Abdihakim Burale **Signature:**  **Date:** 05/05/2026

Acknowledgement

I would like to express my sincere gratitude to all those who supported and guided me throughout this project, which focused on investigating LLVM optimisations for energy-efficient code via software pipelining.

First and foremost, I would like to thank my advisor, **Oswaldo Cadenas**, for his guidance, constructive feedback, and support throughout the development of this project. His advice helped shape the direction of the work and supported the refinement of the experimental approach.

I am also grateful to **Omar Ahmed**, whose professional experience as a compiler engineer provided valuable technical insight during the project. His advice and perspective helped strengthen my understanding of compiler behaviour and the practical challenges involved in analysing optimisation effects.

I would also like to thank **Ian Mackie**, Dean of Engineering, for his leadership and support within the School of Engineering, and **Mike Child**, the module leader, for his guidance and support throughout the module.

Finally, I would like to thank my family, friends, and colleagues for their encouragement, patience, and support during the completion of this project. Their support was greatly appreciated throughout the more challenging stages of the work.

Abstract

This project focuses on the development and evaluation of an experimental framework for measuring the energy impact of LLVM compiler optimisations on loop-intensive programs, with particular emphasis on software-pipelining-related optimisation behaviour. The primary goal is to investigate whether compiler optimisation can reduce total energy consumption when programs are executed on real low-power hardware, rather than relying only on simulation-based evaluation. This is relevant because energy efficiency is increasingly important in embedded systems, mobile devices, data centres, and other computing environments where performance, power use, and operational cost are closely connected. The study compares selected LLVM optimisation configurations in terms of runtime, average power draw, and estimated energy consumption. The experimental method proceeds systematically from benchmark selection and binary compilation to automated execution, PMIC-based power logging, CSV data collection, and numerical analysis. Four loop-intensive PolyBench/C kernels, namely 2mm, 3mm, gemm, and corr, were compiled using LLVM/Clang 18.1.0 on a Raspberry Pi 5. Each benchmark was built using O0, O2, O3, and O3_no_vec, where O3_no_vec disables loop and SLP vectorisation while retaining aggressive optimisation. The binaries were then executed repeatedly under controlled conditions, with system-level voltage, current, and power readings collected through the Raspberry Pi 5 PMIC interface. The collected CSV logs were processed to calculate mean runtime, average power, and estimated energy consumption for each benchmark and optimisation variant. Assembly output was also inspected to identify low-level differences in generated loop structure, instruction patterns, vector-register use, and fused multiply-add operations. The key findings show that all optimised configurations reduced estimated energy consumption compared with the unoptimised O0 baseline. Although optimised binaries generally increased average power draw during execution, their reduction in runtime was large enough to reduce total estimated energy. Across the tested configurations, LLVM optimisation produced an average estimated energy reduction of approximately 54.8% relative to O0, with the strongest improvement observed in gemm, where O3 reduced estimated energy by approximately 82.8%. The O3 and O3_no_vec comparison showed that optimisation effects were workload dependent, as disabling vectorisation increased energy for gemm, 2mm, and corr, but slightly reduced energy for 3mm. Assembly inspection supported these findings by showing that the optimised gemm code used vector registers, fused multiply-add instructions, and a more compact loop body than the O0 version. However, the results do not prove that software pipelining alone caused the energy reductions, since O2 and O3 apply several compiler transformations at the same time, including vectorisation, instruction scheduling, register allocation, and loop restructuring. Overall, the project demonstrates that LLVM loop-level optimisation can improve estimated energy efficiency on real low-power hardware, while also showing the need for cautious interpretation when attributing energy savings to individual compiler techniques.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF LISTINGS

1 Introduction

1.1 Project Background

The field of Information and Communication Technology (ICT) has become a major contributor to global electricity consumption. This growth of the energy consumption is reflected in earlier projections of global connectivity, with research estimating that by 2020 the number of Internet-connected devices would reach between 25 and 50 billion.(Mahdavinejad et al., 2019). Information and Communication Technology, including data centres, communication networks and user devices, accounted for an estimated 4-6% of global electricity use in 2020. (Ross and Christie, 2024)

Given this trend, energy optimisations have become a fundamental design priority. Studies show that while pure hardware-only or software-only optimisation techniques each provide relatively modest improvements, research demonstrates that the greatest improvement arises when both are applied together, yielding an average energy savings of 64%. (Esakkimuthu et al., 2000)

Within the software stack, compilers are a key mechanism for controlling energy consumption. As programs are translated from high-level languages into machine code , they pass through a series of compiler phases that apply transformations influencing execution speed, resource usage and overall program behaviour. Because these transformations determine the final low-level instructions executed by the processor, they also have a direct impact on a program's energy consumption. However, while many compiler optimisations are designed to improve performance, their precise effects on energy usage remains less well understood, and performance gains do not necessarily translate to energy savings.

LLVM provides a suitable platform for exploring these issues due to its modular design and support for a wide range of programming languages and hardware architectures. Its platform-independent, Static Single Assignment (SSA) based intermediate representation enables fine-grained control over optimisation passes, making it well suited for experimental analysis of compiler behaviour. Originally developed as a research project, LLVM has since evolved into a widely adopted compiler infrastructure in both academia and industry (Zhao et al., 2012).

Modern compilers apply a wide range of optimisation techniques, such as loop unrolling, register allocation, instruction scheduling, and software pipelining, to improve execution efficiency and resource utilisation. Software pipelining is particularly relevant for loop-intensive code, as it aims to increase instruction-level parallelism by overlapping operations from different loop iterations (Allan et al., 1995). By improving how loop execution is scheduled, software pipelining has the potential to enhance processor utilisation and influence both performance and energy behaviour.

1.2 Problem Statement

Despite the widespread use of compiler optimisations to improve program performance, their impact on overall energy consumption remains unclear, particularly for advanced loop transformations such as software pipelining. While software pipelining is well known to increase instruction-level parallelism, existing research suggests that performance improvements do not always correspond to energy savings. Furthermore, much of the prior work relies on simulation-based evaluation rather than measurements taken from real hardware.

This leads to the central research question of this project: to what extent does software pipelining within the LLVM compiler framework reduce or increase total system energy consumption when executed on real, low-power hardware?

1.3 Project Aims

The aim of this project is to investigate LLVM's optimisation techniques influence program energy consumption, with a particular focus on the software pipeline technique. The system will measure and compare energy use and performance on real hardware to determine whether software pipelining leads to genuine energy efficiency.

1.4 Project Values

Energy efficiency is important for both individual consumers and businesses, as billions of devices worldwide rely on low-power operation (Mahdavinejad et al., 2019). Large technology companies also operate energy-intensive data centres to support digital services, and the cost of running this infrastructure is heavily influenced by the power and cooling demands for clients. Increasing the energy efficiency of data centers is therefore essential to limit energy consumptions, thereby reducing operational cost. (Ahmed, Bollen and Alvarez, 2021)

1.5 Objectives

- To critically analyse current research on compiler-level energy optimisations, with a specific focus on loop transformation techniques.
- To investigate the specific impact of Software Pipelining within the LLVM framework on both energy consumption and execution performance
- To benchmark and compare the energy efficiency of Software pipelining against other standard loop optimisations to identify performance-energy trade-offs.

1.6 Project Interpretation and Design Decisions

This project interprets the original proposal as an experimental investigation into the energy impact of compiler-level optimisations on real hardware, rather than simulation-based evaluation. LLVM was selected due to its modular optimisation pipeline and fine-grained control over individual optimisation passes, which enables controlled experimentation. The scope of the project is deliberately limited to loop-centric optimisations, with software pipelining chosen as the primary focus due to its strong performance benefits and unclear energy implications in existing research. The evaluation targets low-power ARM-based systems using system-level energy measurements to reflect realistic deployment conditions, rather than fine-grained per-component energy modelling.

1.7 Commercial and Economic Insights

Beyond the technical motivation, this project also has significant commercial and economic relevance, particularly in large-scale and cost-sensitive computing environments.

Beyond environmental gains, energy efficiency delivers essential commercial and economic growth opportunities for the ICT sector. The key benefits include:

- **Scale and Competitiveness in Cloud & HPC Markets:** Energy-efficient compiler optimisations directly affects operational scalability for cloud providers and HPC (High Performance Computing) centres. By reducing CPU cycles and power drawn per workload, providers are able to serve more customers with the same hardware footprint. The success of cloud computing has created large-scale infrastructures that are costly to operate, and power supply is identified as a dominant component of operational expenditure in these environments *(Laganà et al., 2018)* .This economic pressure drives cloud operators to pursue efficiency improvements across the software stack, including at the compiler level.

- **Profitability Across Compute Driven Sectors:** For scientific and HPC workloads, faster and more predictable execution reduces the number and type of compute instances researchers must purchase, lowering total simulation costs due to cloud cost-performance trade-offs (Taufer and Rosenberg, 2016). Because cloud platforms bill according to the resources and time consumed, having faster compilation and execution times reduces the overall expenditure by shortening the total runtime and optimises the resource usage.

1.8 Legal, Social, Ethical, and Professional Issues

There are several important concerns when applying compiler optimisations, as these techniques change how programs behave and execute. This means that they can introduce legal obligations, ethical challenges, social effects and professional responsibilities. For example, compiler optimisations may preserve the functional correctness of a program but can still break the security properties assumed by developers, creating a “correctness-security gap” that has legal and professional implications for systems where security is critical (D’Silva, Payer and Song, 2015)

Another important concern is ethics, particularly the responsibility to minimise unnecessary energy consumption and to avoid optimisation choices that lead to inefficient or misleading system behaviour. Research on embedded systems shows that compiler optimisations affect not only performance but also the overall power usage of a program, and that generating efficient binaries is essential for achieving more energy efficient systems (Daud, Ahmad and Murhty, 2008)

Compiler optimisations also have social implications. Because compilers work invisibly in the background, their impact is often overlooked; however, with billions of devices in use every day (Mahdavinejad et al., 2019), more efficient software can reduce overall energy demand. This supports wider sustainability goals and lowers the environmental impact of digital technology.

2 Literature Review

2.1 Overview of Compiler Optimisations and Energy Efficiency

Historically, compiler optimisation research has focused on improving execution performance. Early and foundational studies concentrated on reducing execution time and making better use of the available hardware resources, shaping how standard optimisation techniques and optimisation levels are designed. As a result, energy efficiency has often been treated as a secondary consideration in the literature.

In recent years, concerns about power consumption in embedded, mobile, and large-scale computing systems have increased in energy-aware compilation. Researchers have begun to explore how compiler behaviour affects energy usage, particularly in low-power and resource-constrained environments. However, much of the existing literature still evaluates compiler optimisations primarily in terms of performance, and the energy effects of many optimisation techniques remain less well understood.

2.2 Loop Optimisations and Software Pipelining

Loops are an important target of compiler optimisation, as they often account for a large portion of program execution time and present opportunities for performance improvement. Consequently, loop optimisations have been widely studied in compiler research over several decades. A range of techniques has been proposed to restructure loops in order to improve parallelism and execution behaviour, particularly in high-performance hardware (Pugh, 1991).

Among these loop-level techniques, software pipelining is commonly used to improve performance. However, the impact of software pipelining on energy consumption is far less understood. Prior work suggests that “power savings and energy savings may not go hand-in-hand”, and that most optimisations apart from loop unrolling can even increase nergy usage in the processor core (Kandemir et al., 2000). This shows that faster execution does not always mean lower energy use. In some cases, an optimisation may reduce execution time while still increasing energy consumption. As a result, focusing only on performance improvements may overlook important aspects of energy consumption.

2.3 Energy Evaluation Approaches in Compiler Research

How energy is evaluated in existing studies also contributes to this limited understanding. Many studies on compiler optimisations rely on simulation-based energy models rather than measurements taken from real hardware. These models are useful for controlled experiments, but they do not always reflect how modern processors behave in practice. For example, Kandemir et al. (2000) estimate energy consumption using a cycle-accurate simulator, which cannot capture all aspects of real hardware behaviour.

While simulation-based approaches do make it easier to compare optimisation techniques in a controlled setting, they often simplify or in some cases omit factors such as system-level power management, memory behaviour, and background operating system activity. As a result, the energy estimates produced by these models may differ from energy consumption observed when programs are executed on real hardware.

2.4 System-Level vs Component-Level Energy Analysis

In addition, many existing studies focus on individual parts of the system, such as the processor core or cache, rather than measuring total system energy. While this component-level analysis is useful for understanding how specific optimisations affect particular hardware structures, it provides limited insight into how the system behaves as a whole. By examining components in isolation, these studies can overlook interactions between different parts of the system, including the processor, memory, and other subsystems. As a result, it can be difficult to determine the overall energy impact of compiler optimisations, such as software pipelining, when they are deployed on real devices.

2.5 Critical Assessment

The literature reviewed provides a broad overview of compiler optimisations and their impact on performance and energy consumption. However, much of this work places limited emphasis on critically examining the challenges of energy-aware compilation. Although performance-oriented techniques such as loop transformations and software pipelining are widely discussed, their potential disadvantages in terms of energy consumption are often underexplored.

In particular, many studies assume that improvements in execution time will naturally lead to reductions in energy consumption, despite evidence suggesting that this relationship does not always hold. Additionally, a significant portion of the literature relies on simulation-based or component-level energy analysis, which may not accurately capture system-level behaviour on real hardware. As a result, interactions between the processor, memory, and other subsystems are often insufficiently represented in existing evaluations.

2.6 Research Gaps

Despite increasing interest in energy-aware compiler optimisation, several important gaps remain in the existing literature. Much of the prior work has focused on improving execution performance, with energy consumption often treated as a secondary concern. Even when energy is considered, analyses frequently rely on simplified models or examine individual system components in isolation.

In particular, loop-level optimisations such as software pipelining are well studied from a performance perspective, yet their impact on total energy consumption remains insufficiently explored. While prior work, such as Kandemir et al. (2000), highlights that energy and performance improvements do not necessarily align, there is limited empirical evidence demonstrating how software pipelining affects overall system energy when executed on real hardware.

Consequently, there is a lack of system-level experimental studies that measure both performance and energy consumption for software pipelining on real devices. Addressing this gap would provide clearer insight into the true energy implications of compiler optimisations and support the development of more energy-aware optimisation strategies.

3 Technical Review

This technical review explores the technologies that can be used to investigate compiler-level energy optimisation. Since the project focuses on how software pipelining affects energy consumption, it looks at different compiler frameworks, hardware platforms, and energy measurement approaches that support this kind of analysis. The review discusses why LLVM and low-power ARM-based hardware are appropriate choices, while also considering alternative options and their limitations.

3.1 LLVM Compiler Framework

LLVM is a modular compiler framework built around a language-independent intermediate representation (IR) in Static Single Assignment (SSA) form. It was designed to work well with modern programming languages and hardware, while supporting advanced optimisation techniques. As Lattner explains, *"the Low-Level Virtual Machine (LLVM) [is] a compiler infrastructure which is well-suited for modern programming languages and architectures"* (Lattner, 2002). LLVM applies optimisations through a series of configurable passes, which makes it possible to control and study individual transformations such as loop unrolling, vectorisation, and software pipelining.

One of LLVM's main strengths is the fine-grained control it provides over optimisation passes. This makes it possible to enable or disable individual loop transformations, allowing specific optimisations such as software pipelining to be studied in isolation. LLVM is also widely used in both academic research and industry, and offers strong support for modern architectures, including ARM-based systems.

Despite these advantages, LLVM is a large and complex codebase, which can make it more difficult to learn and configure. In addition, the behaviour of optimisation passes may vary between LLVM versions, which can affect reproducibility if compiler versions are not carefully controlled.

Overall, LLVM is well suited to this project because it provides the flexibility needed to isolate loop-level optimisations while supporting controlled experimentation on real hardware.

3.2 GNU Compiler Collection

GCC is a long-established and widely used compiler that supports many programming languages and hardware platforms. It includes a large number of optimisation techniques and is often used as a baseline in performance studies. Because it has been developed over many years, there is extensive documentation and a large user community available.

However, compared to LLVM, GCC offers less direct control over individual optimisation passes. Although it applies aggressive optimisations, it is harder to selectively enable or disable specific loop transformations. This makes it more difficult to study the effect of a single optimisation, such as software pipelining, on energy consumption in isolation.

For these reasons, while GCC is a strong and reliable compiler, it is less well suited to the aims of this project. LLVM/Clang was therefore chosen because it provides the level of control needed to analyse the energy and performance impact of individual loop-level optimisations.

3.2.1 Comparison of LLVM/Clang and GCC

The table below shows a comparison between LLVM/Clang and GCC. It compares features that are relevant to measuring the energy impact of compiler optimisations.

TABLE 1: COMPARISON OF LLVM/CLANG AND GNU GCC FOR COMPILER OPTIMISATION RESEARCH

Feature	LLVM/Clang	GNU GCC
Compiler architecture	Modular, multi-stage framework with explicit IR	Monolithic compiler pipeline
Front end	Clang (C/C++, etc.)	GCC front ends (C, C++, etc.)
Intermediate representation	SSA-based, language-independent LLVM IR	Multiple internal IRs, less exposed
Control over optimisation passes	Fine-grained control; individual passes can be enabled or disabled	Limited fine-grained control over individual passes
Support for loop-level optimisations	Strong support, including software pipelining and advanced loop transforms	Strong support, but harder to isolate specific loop passes
Suitability for experimental research	High; designed to support research and experimentation	Moderate; optimised for production compilation
Energy optimisation research support	Widely used in academic energy and performance studies	Less commonly used for fine-grained energy studies
Target architecture support	Strong support for ARM and embedded systems	Strong support across many architectures
Toolchain maturity	Mature and actively developed	Very mature and widely deployed
Relevance to this project	**Selected platform** due to control and transparency	Considered but not selected

4 Methodology

4.1 Research Approach

This project follows an experimental methodology to evaluate the energy impact of LLVM optimisation techniques. The approach involves compiling a set of loop-intensive benchmark programs using different optimisation configurations, executing the resulting binaries on real hardware, and recording power and performance data for analysis.

The experiment compares multiple PolyBench kernels across multiple compiler optimisation configurations. For each kernel, the optimisation settings are varied while hardware configuration, input size, and execution environment are kept constant. This allows differences in execution time and system-level power consumption to be compared under controlled conditions. These measurements are then used to estimate total energy usage.

A repeated-measures design is used, where each benchmark configuration is executed multiple times under the same conditions. This reduces the effect of system noise and runtime variation, and increases the reliability of the results by ensuring that observed differences are more likely to reflect compiler behaviour rather than random fluctuations.

4.2 Methodological Rationale

A quantitative approach is used because the study compares numerical differences in execution time and energy consumption across compiler optimisation configurations. The aim is to measure and compare system behaviour under controlled conditions rather than interpret it qualitatively.

An experimental method is preferred over simulation-based approaches because simulation does not fully capture real hardware effects such as frequency scaling, cache behaviour, and operating system interference. These factors are important in this study because the results are intended to reflect measured behaviour on physical hardware.

An observational approach is not suitable because it would not allow direct control over compiler optimisation settings. Without controlling this variable, it would not be possible to compare the effect of selected LLVM optimisation configurations on energy consumption. For these reasons, an experimental quantitative approach is the most suitable method for answering the research question.

4.3 Benchmark Suite: PolyBench/C

PolyBench was used as the benchmark suite for the experiments. PolyBench is a collection of loop-intensive benchmark programs written in C, widely used in compiler and performance research. It was selected because the programs contain regular loop structures and numerical kernels that are suitable for studying the effect of compiler optimisations, particularly loop-related transformations.

The selected kernels included:

- **2mm** - Two matrix multiplications perform sequentially.
- **3mm** - Three matrix multiplications perform sequentially.
- **gemm** - General matrix multiplication.
- **corr** - Correlation computation.

Other PolyBench kernels, such as gemver, gesummv, syr2k, and syrk, were not included in the final benchmark set in order to keep the study manageable while still covering a good range of loop-intensive workloads.

A strong reason for selecting PolyBench is that it is a well-established benchmark suite in compiler and performance research, making it a suitable and credible basis for experimental evaluation. It provides loop-intensive programs that are widely used for this type of study, which supports fairer comparison and strengthens the academic validity of the results. PolyBench also includes configurable problem size macros, allowing the benchmarks to be built with workloads that are large enough to produce measurable runtime and power data.

Although developing custom benchmark programs would have offered more direct control over the code, such programs would have been less established and narrower in scope. For this reason, PolyBench was the more suitable choice for this project.

5 Selection of Optimisation Techniques

This study compares selected LLVM optimisation configurations to examine how compiler-generated changes affect runtime and energy consumption. Particular attention is given to loop-related behaviour, including instruction scheduling, vectorisation, and software pipelining, because these optimisations can influence how efficiently loop-intensive programs execute on the Raspberry Pi 5.

The compiler provides a range of optimisation levels, from little or no optimisation at -O0, which is used as a baseline, to more aggressive settings such as -O2 and -O3. These optimisation levels apply compiler passes that may affect loop structure, instruction scheduling, vectorisation, register allocation, and generated code efficiency.

An additional configuration, labelled O3_no_vec in this report, was also included. This configuration uses -O3 -fno-vectorize -fno-slp-vectorize, meaning that aggressive optimisation is still enabled while loop vectorisation and SLP vectorisation are disabled. This provides a comparison point for examining whether observed runtime and energy differences are mainly linked to vectorisation or to other optimisation behaviour, including scheduling and loop execution effects.

TABLE 2: LLVM OPTIMISATION CONFIGURATIONS USED IN THE EXPERIMENT

Configuration label	Configuration	Main Purpose	Likely effects on generated code
O0	-O0	Minimal	Baseline for unoptimised code
O2	-O2	Medium-High	Stronger loop and scheduling optimisation
O3	-O3	Aggressive	Most likely to show advanced optimisation effects
O3_no_vec	-O3 -fno-vectorize -fno-slp-vectorize	Aggressive without vectorisation	Helps isolate non-vectorisation optimisation effects

6 System Platform

6.1.1 Hardware Platform

The experiments were conducted on a Raspberry Pi 5, chosen for its low-power ARM architecture and accessible built-in power monitoring features. This made it a suitable platform for controlled energy measurement experiments. The Raspberry Pi 5 includes a Power Management Integrated Circuit (PMIC) that provides system-level voltage and current readings during program execution. This allows power behaviour to be monitored directly on the device without requiring external measurement hardware.

TABLE 3: HARDWARE SPECIFICATION USED FOR THE EXPERIMENTAL PLATFORM

Component	**Specification**
Hardware platform	Raspberry Pi 5
Processor	ARM Cortex-A76
Memory	8GB LPDDR4
Operating system	Raspberry Pi OS (64-bit)
Measurement source	PMIC-reported system power values

Compared with alternative platforms such as desktop systems, laptops, or microcontroller-based boards, the Raspberry Pi 5 offered the most suitable balance for this study. Desktop and laptop systems provide greater computational power, but they also introduce higher system complexity, less predictable background activity, and more difficult power measurement setups, which would make controlled comparison harder. At the other end of the scale, smaller embedded platforms such as Arduino-class boards are simpler and lower cost, but they do not provide a realistic environment for running LLVM-compiled PolyBench workloads at the level required for this investigation. The Raspberry Pi 5 sits between these extremes. It is powerful enough to execute the benchmark suite and support the LLVM toolchain, while still remaining relatively low cost, accessible, and easier to control than a full desktop platform. For this reason, it provided the best practical balance between realism, simplicity, measurement access, and experimental repeatability for the aims of this project.

6.1.2 Software Environment

The software environment used in this project was designed to support controlled and repeatable experimentation. It consisted of Raspberry Pi OS (64-bit), LLVM/Clang 18.1.0, the PolyBench benchmark suite, and Python scripts used for automation and data logging. Together, these components provided a stable environment for compiling benchmark programs, executing them under consistent conditions, and recording energy and performance data on the Raspberry Pi 5.

6.1.3 Operating System: Raspberry Pi OS (64-Bit)

Raspberry Pi OS (64-bit) was used as the operating system for all experiments. This distribution was selected because it is the standard Linux environment for Raspberry Pi hardware and provides strong compatibility with the device's drivers, systems, and development tools. It also offers a stable platform for compiling programs, running Python automation scripts, and collecting measurement data under controlled conditions.

This choice of Raspberry Pi OS supported the experimental reliability in two ways. First, it reduced setup complexity by using a platform that is closely aligned with the target hardware. Secondly, it improved reproducibility by relying on a widely supported and well-documented software environment with the Raspberry Pi ecosystem.

Alternative Linux distributions such as Ubuntu could also have been used, and these provide a strong general-purpose development environment. However, this project did not require a broader desktop platform or cross-platform feature set, since the main aim was to minimise unnecessary complexity and maintain a stable test environment. For this reason, Raspberry Pi OS was selected as the most appropriate operating system for the experimental platform.

6.1.4 Compiler Toolchain: LLVM/Clang 18.1.0

LLVM/Clang 18.1.0 was used to compile all benchmark programs. It was chosen because this project investigates the energy impact of LLVM optimisation behaviour, especially loop optimisations and software pipelining. Using LLVM therefore ensured that the compiler toolchain matched the focus of the research.

A further reason for this choice was the level of control available during compilation. LLVM supports both standard optimisation levels and more targeted compiler settings, which made it possible to produce multiple benchmark variants under consistent build conditions. This was important for fair experimental comparison.

An alternative compiler such as GCC could also have been used, but it was less appropriate for this study. The aim was not to compare different compiler families, but to examine optimisation behaviour within the LLVM framework itself. For that reason, LLVM/Clang was the most appropriate choice.

6.1.5 Automation Scripts: Python

Python was used to implement the automation scripts for compilation, execution control, and data logging. It was selected because it is well suited to scripting experimental workflows, with clear syntax and strong support for file handling, process control, and CSV generation. This made it practical for coordinating repeated benchmark runs and managing the large number of generated outputs.

Python was also a suitable choice because it allowed the build and measurement workflow to be developed quickly and adapted as the experimental setup evolved. Lower level languages such as C or C++ would have added unnecessary complexity for orchestration tasks, while shell scripting alone would have been less suitable for managing structured logging and multi-stage control logic. For this reason, Python provided the best balance of simplicity, flexibility, and maintainability.

7 Requirements

7.1 System Definition and Boundary

The system developed in this project is an experimental energy evaluation framework designed to measure and compare the runtime and energy consumption of selected LLVM optimisation configurations. The framework uses loop-intensive benchmark programs to investigate whether LLVM optimisation behaviour, including loop execution, scheduling, vectorisation, and software-pipelining-related effects, may contribute to energy-efficiency benefits. The framework operates by compiling benchmark programs under controlled optimisation configurations, executing the resulting binaries on real hardware, collecting system-level power measurements, and calculating total energy usage.

The system boundary includes:

- Benchmark source programs (loop-intensive C programs)
- LLVM/Clang compiler toolchain (version 18.1.0)
- Raspberry Pi 5 hardware platform
- Power measurement interface using the on-board PMIC
- Data logging and energy computation mechanisms

The system does not include modification of LLVM's internal optimisation algorithms. It uses existing optimisation passes and evaluates their external energy effects. It also does not include hardware-level instrumentation beyond PMIC-reported system power readings.

The primary stakeholder is the researcher conducting controlled experimental analysis. Secondary stakeholders include academic assessors evaluating methodological rigour and reproducibility.

7.2 Requirements Elicitation and Analysis Approach

The system requirements were derived using a research-driven elicitation approach rather than traditional client-based elicitation. Requirements emerge from four primary sources:

TABLE 4: SOURCES USED TO DERIVE SYSTEM REQUIREMENTS

Source	**Influence on requirements**
Research Question	Required controlled measurement of runtime and energy across optimisation configurations.
Literature gaps	Supported the need for benchmark-based comparison, repeated runs, and structured logging.
Hardware constraints	Limited measurement granularity to system-level PMIC readings.
Experimental methodology standards	Required controlled variables, repeatability, structured data collection, and reproducible configurations.

The requirements were categorised into:

- Functional Requirements
- Non-Functional Requirements
- System requirements
- Explicit Exclusions

This structured classification ensures clarity, traceability, and alignment with the project aims

7.3 Functional Requirements

These are the functional requirements needed for the system to run the experiment. This includes compiling benchmark programs, executing them on the Raspberry Pi 5, collecting power and runtime data, and producing results for analysis. These functions define the core steps of the experimental process.

The requirements are specified using requirement identifiers, priority, and rationale. This makes each requirement traceable to the project aim and allows the final artefact to be checked against the intended design.

TABLE 5: FUNCTIONAL REQUIREMENTS FOR THE EXPERIMENTAL FRAMEWORK

ID	Requirement	Priority	Rationale
FR1	The framework shall compile selected benchmark programs using LLVM/Clang 18.1.0.	Must	Required to evaluate LLVM optimisation behaviour using the selected compiler toolchain.
FR2	The framework shall support selected LLVM optimisation configurations.	Must	Required to compare runtime and energy behaviour across different compiler settings.
FR3	The framework shall execute compiled benchmark binaries on the Raspberry Pi 5.	Must	Required to collect runtime and power data on real hardware.
FR4	The framework shall run each benchmark configuration more than once.	Must	Required to reduce the effect of random runtime variation and measurement noise.
FR5	The framework shall support stabilisation and rest periods between measured executions.	Should	Required to reduce the effect of early system variation and carry-over effects between runs.
FR6	The framework shall capture PMIC voltage, current, and power readings during benchmark execution.	Must	Required to estimate power and energy consumption.
FR7	The framework shall record benchmark name, optimisation configuration, run number, timing data, and power readings.	Must	Required to make each run traceable and comparable.
FR8	The framework shall export experiment results in a structured CSV format.	Must	Required for later processing, comparison, and analysis.

7.4 Non-Functional Requirements

TABLE 6: NON-FUNCTIONAL REQUIREMENTS FOR THE EXPERIMENTAL FRAMEWORK

ID	Requirement	Priority	Rationale
NFR1	The experiment shall use a fixed hardware platform, operating system, compiler version, and benchmark input size.	Must	Required to support reproducibility and fair comparison between optimisation configurations.
NFR2	Benchmark execution shall be repeatable under the same hardware and software conditions.	Must	Required to improve the reliability of the experimental results.
NFR3	The measurement process shall minimise unnecessary background activity and logging overhead.	Should	Required to reduce measurement noise and avoid distorting benchmark runtime.
NFR4	Data logging shall use a consistent format across all benchmarks runs.	Must	Required to allow reliable comparison between optimisation configurations.
NFR5	The collected data shall be suitable for statistical analysis and comparison.	Must	Required to calculate averages, compare configurations, and identify variation between runs.
NFR6	The framework shall remain manageable within the project timeframe.	Should	Required to keep the number of benchmarks and optimisation configurations realistic for a dissertation project.

7.5 System Requirements

These are the system requirements needed for the experimental setup to run correctly. They include the hardware platform, software environment, and benchmark data required to carry out the experiments and collect consistent results.

TABLE 7: SYSTEM REQUIREMENTS FOR THE EXPERIMENTAL SETUP

Category	Requirement
Hardware	Raspberry Pi 5 with ARM-based architecture
Hardware	Minimum 8GB RAM
Hardware	Stable power supply
Hardware	Access to PMIC readings
Software	Raspberry Pi OS (64-bit)
Software	LLVM/Clang 18.1.0
Software	Python
Software	Terminal/SSH environment
Data	Loop-intensive benchmark programs
Data	Controlled input sizes
Data	Output data including execution time, power readings, and energy values
Data	Structured storage format, such as CSV

7.6 Explicit Exclusions

The system is limited to evaluating existing LLVM optimisation configurations and does not modify the compiler itself. Although the study investigates optimisation behaviour related to loop execution and software pipelining, it does not implement new LLVM passes or directly alter LLVM's internal optimisation algorithms. Energy measurement is restricted to system-level PMIC readings from the Raspberry Pi 5, so no external power measurement hardware or fine-grained component-level instrumentation is included.

The evaluation focuses on selected loop-intensive benchmarks and selected optimisation configurations. Results are specific to the Raspberry Pi 5 and are not intended to generalise directly to all hardware platforms, processor architectures, or workloads.

8 Project Planning and Design Evolution

8.1 Detailed Implementation Strategy

The implementation of the artefact is organised into a week-by-week strategy. This structure provides a clear order of work for developing the experimental framework, beginning with environment setup and ending with analysis and dissertation refinement

Each stage builds on the previous one so that the hardware, software, benchmark programs, automation scripts, and data analysis process can be tested progressively.

- **Week 11 - Environment Setup and Measurement Validation**
    - Configure the Raspberry Pi 5 with Raspberry Pi OS, LLVM/Clang 18.1.0, Python, and the required development tools.
    - Confirm that C programs can compile and run successfully, and validate that PMIC voltage, current, and power readings can be captured through the Raspberry Pi power-monitoring interface during execution.

- **Week 12 - Initial Benchmark Testing and Design Revision**
    - Test a simple custom C99 loop-based program to validate the basic compilation, execution, and measurement workflow.
    - Review the limitations of using a single custom benchmark and revise the design to use selected PolyBench kernels for improved workload variety, academic suitability, and comparability with compiler research.

- **Week 13 - PolyBench Preparation and Optimisation Configuration Design**
    - Prepare and test selected PolyBench kernels, including 2mm, 3mm, gemm, and corr, to confirm that they compile, execute correctly, and produce suitable runtime behaviour for measurement.
    - Define the LLVM optimisation configurations, including `-O0`, `-O2`, `-O3`, and -O3_no_vec, and confirm that they compile consistently across the selected benchmarks.

- **Week 14 - Build and Logging Script Development**
    - Develop the build script to compile each selected benchmark under each optimisation configuration and organise the generated binaries by benchmark name and optimisation variant.
    - Develop the logging script to record PMIC voltage, current, and power readings, along with benchmark metadata, run numbers, timing information, and structured CSV output.

- **Week 15 - Controller Script and Workflow Test**
    - Develop the controller script to coordinate benchmark execution, select the correct binaries, start and stop logging, and run each optimisation configuration in a fixed order.
    - Integrate repeated executions, stabilisation runs, and rest periods between measured runs to improve consistency and reduce carry-over effects.

- **Week 16 - Experimental Validation and Trial Runs**
    - Run trial executions on a small subset of benchmarks and optimisation configurations to confirm that the build, controller, and logging scripts work together correctly.
    - Check generated CSV files for missing values, incorrect labels, abnormal readings, failed executions, and timing inconsistencies before carrying out the full experiment.

- **Week 17 - Data Processing and Analysis**
    - Process the collected CSV files to calculate runtime, average power, and estimated energy consumption for each benchmark and optimisation configuration.
    - Compare results across optimisation levels, check the consistency of repeated runs, identify any outliers requiring explanation, and prepare tables or graphs for the results section.

- **Week 18 to Week 20 - Dissertation Writing and Final Revision**
    - Update the dissertation sections to reflect the final artefact, experimental process, results, analysis, limitations, and evaluation.

8.1.1 GANTT Chart

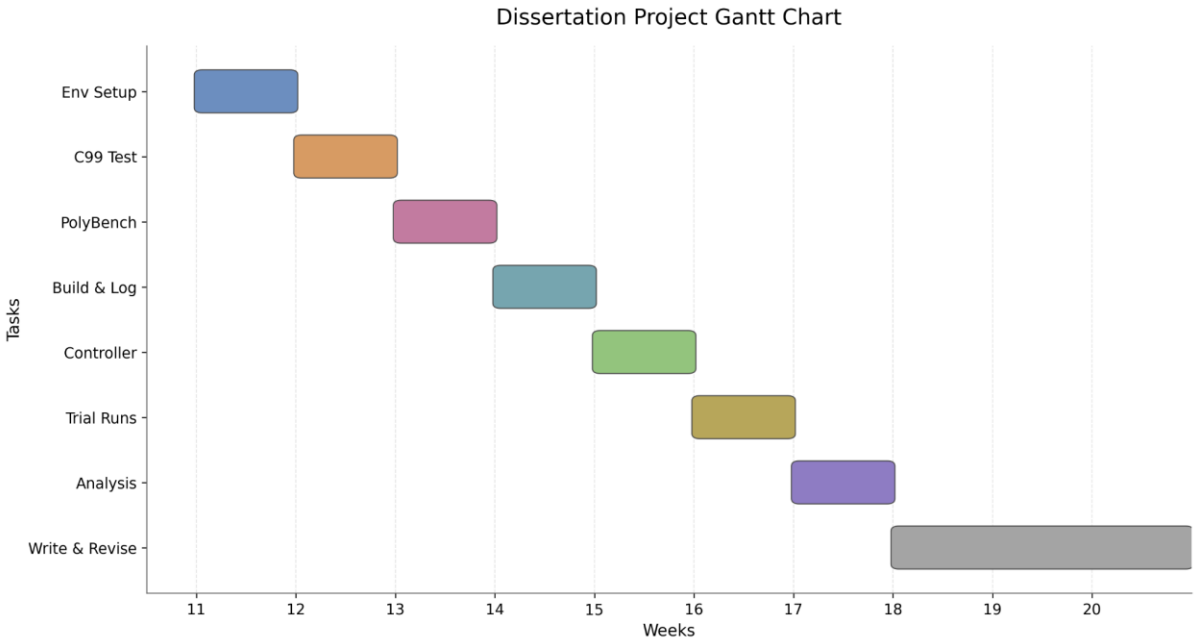


FIGURE 1: GANTT CHART SHOWING THE PLANNED PROJECT SCHEDULE

9 Evolution of the Experimental Design

The experimental design was refined as the practical and methodological needs of the project became clearer. The initial plan was to write a custom C99 benchmark program with a large number of loop iterations, such as a million-loop workload, and use this to compare LLVM optimisation configurations. This would have given direct control over the source code and workload. However, it was later judged to be too narrow, as a single custom program would have limited comparison with existing compiler research and may have produced results specific to one artificial workload.

Following advisor feedback and further research into benchmark-based compiler evaluation, the design was revised to use selected PolyBench kernels. This improved the academic suitability of the experiment because PolyBench provides established loop-intensive C programs commonly used in compiler and performance research. It also allowed optimisation behaviour to be tested across more than one workload while keeping the project manageable.

The execution procedure was also revised during the design process. The original plan did not fully account for repeated runs, stabilisation, or rest periods between executions. Further consideration of experimental reliability showed that single benchmark executions could be affected by system noise, temporary background activity, caching effects, or thermal variation.

The final design therefore uses repeated runs for each benchmark and optimisation configuration. A stabilisation run is included before measured runs, and rest periods are used between executions to reduce carry-over effects. This revision strengthened the reliability of the experiment by making the results less dependent on a single execution.

9.1.1 Validity, Reliability, and Limitations

The validity of this study is supported by keeping the hardware platform, software environment, compiler version, benchmark input size, and execution procedure consistent across all experiments. Each benchmark was compiled and executed using the same baseline settings, so differences in runtime, average power, and estimated energy consumption can be linked more directly to the optimisation configurations being tested rather than to uncontrolled changes in the experimental setup.

Reliability was improved by running each benchmark configuration more than once. During the design process, the experiment was changed from a single-run approach to a repeated-run structure because one execution may be affected by operating system activity, runtime variation, or short-term changes in power use. A stabilisation run and rest periods between executions were also included to reduce the effect of early system variation and carry-over effects between runs. The measured runs were then averaged, providing a more reliable comparison than using only one result.

The design also improves reproducibility by using a fixed LLVM/Clang version, a consistent Raspberry Pi OS environment, defined optimisation flags, selected PolyBench kernels, and structured CSV logging. These choices make the experimental setup easier to repeat and allow the collected data to be checked and analysed systematically.

However, the approach has limitations. Power measurements were taken using the Raspberry Pi 5 PMIC at system level, meaning the study cannot isolate the energy consumption of individual components such as the CPU, memory, or storage. The measurement approach therefore provides estimated system-level energy values rather than precise CPU-only energy measurements.

One implementation limitation is that the compilation scripts used the target flag `-mcpu=cortex-a72`, while the Raspberry Pi 5 uses a Cortex-A76 processor. This means the generated binaries were compiled for a compatible ARM processor target, but were not fully tuned for the exact Raspberry Pi 5 core. The results therefore reflect the tested compiler configuration rather than the maximum possible optimisation for the hardware. Future work should repeat the experiment with a Cortex-A76-specific target configuration.

The results are also specific to the Raspberry Pi 5, the selected PolyBench kernels, and the chosen optimisation configurations. They should therefore not be generalised directly to all hardware platforms, processor architectures, or workloads.

9.1.2 Risk Mitigation Strategy

Several risks were identified during the planning and design of the experiment. One major risk was variation in power measurements, as background system activity, thermal behaviour, and short-term operating system processes could introduce noise into the results. This risk was reduced by minimising unnecessary background processes, using repeated benchmark runs, including rest periods between executions, and calculating average values from the measured results.

A second risk was instability or inconsistency in the LLVM toolchain and power measurement setup. This was addressed by fixing the compiler version, using a consistent Raspberry Pi OS environment, and carrying out initial test runs before the main experiment. These checks helped confirm that benchmark compilation, execution, PMIC measurement, and CSV logging were working as expected.

A further risk was project scope. Testing too many benchmarks, optimisation configurations, or measurement variations would have increased the amount of data beyond what could be analysed effectively within the project timeframe. This risk was managed by limiting the final benchmark set to selected loop-intensive PolyBench kernels and focusing on a realistic number of LLVM optimisation configurations. This allowed the experiment to remain manageable while still addressing the project's research aims.

Another risk was over-attributing the measured energy changes to software pipelining alone. Standard LLVM optimisation levels apply multiple compiler transformations, including vectorisation, scheduling, register allocation, and loop restructuring. This risk was mitigated by including the `O3_no_vec` configuration and by interpreting the results cautiously as evidence of loop-level optimisation behaviour rather than proof that software pipelining was the sole cause of the measured energy differences.

10 System Design

10.1Overview of the Artefact

The software artefact developed for this project is an automated experimental framework for evaluating the runtime and energy impact of selected LLVM optimisation configurations. The artefact is made up of three main scripts: a build script, a controller script, and a logging script. Together, these scripts form a pipeline that compiles PolyBench kernels, executes each benchmark under controlled optimisation settings, records PMIC power readings during execution, and stores the results in structured CSV files for later analysis.

The design separates the build, execution control, and measurement logging into separate parts. This makes the framework easier to manage because each script has a clear role. The build script creates the benchmark binaries, the controller script manages the order of execution and repeated runs, and the logging script records the measurement data. The final output is a set of CSV files containing benchmark details, timing data, PMIC readings, and calculated power or energy values.

The artefact does not include a graphical user interface because it is an experimental automation framework rather than an end-user application. The system is used through command-line Python scripts, with results inspected through the generated CSV files. This is suitable because the priority is repeatable experiment control rather than interactive use.

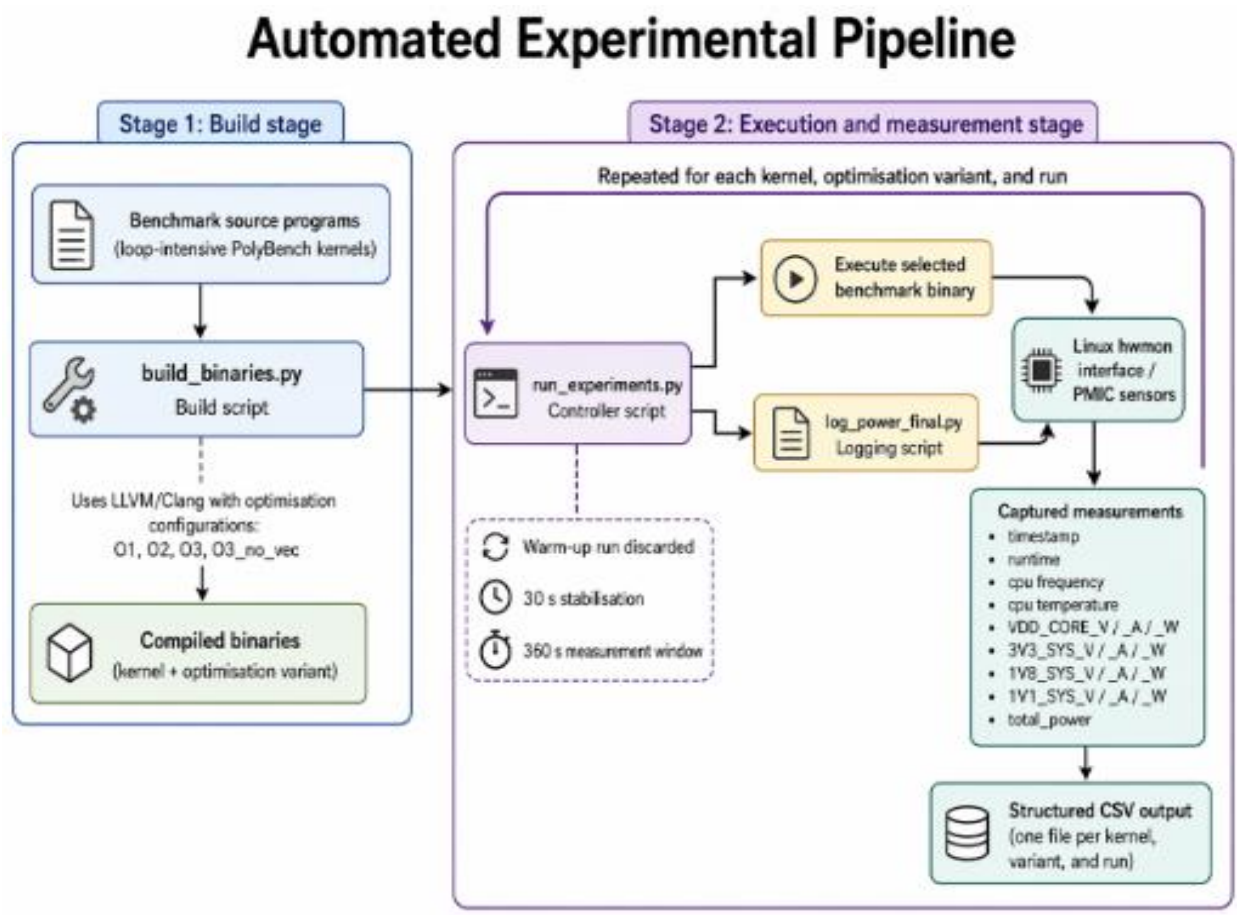


FIGURE 2: HIGH-LEVEL SYSTEM ARCHITECTURE FLOWCHART

Figure 2 shows the high-level architecture of the artefact. The build stage compiles selected PolyBench kernels using LLVM/Clang under defined optimisation configurations. The execution and measurement stage then runs each compiled binary through the controller script, records PMIC readings through the logging script, and stores the resulting runtime and power data in structured CSV files.

10.2 Build Script: build_binaries.sh

The build script is responsible for compiling the benchmark binaries used in the experiment. It applies a consistent set of compiler flags and benchmark configuration settings across all builds, including the selected optimisation variants and the fixed PolyBench data size. The script organises the output into a binaries directory, with separate subdirectories for each benchmark kernel.

Within each kernel directory, multiple executables are generated, each representing a different optimisation configuration. This design ensures that all benchmark variants are built in a consistent and traceable way, so that later differences in runtime or energy behaviour can be attributed to the optimisation settings rather than inconsistent build conditions.

10.3 Controller Script: run_experiments.py

The controller script is responsible for coordinating the full experiment. It selects the compiled binaries, executes each benchmark configuration repeatedly, applies the required timing structure for warm-up, stabilisation, measurement, and delay periods, and invokes the logging process for each recorded run.

It operates across all selected kernels and optimisation variants in a fixed sequence. This design ensures that every experiment follows the same procedure and that the resulting data is collected under consistent conditions.

10.4 Logging Script: log_power_final.py

The logging script is responsible for collecting measurement data during each benchmark run. It reads PMIC power-related values from the Raspberry Pi power-monitoring interface and records them together with timing information, run identifiers, and benchmark metadata in CSV format.

The logged fields include per-rail voltage, current, and power values, as well as aggregate values such as total power and runtime. This design separates measurement capture from experiment control and produces a structured dataset for later analysis.

TABLE 8: PMIC RAILS RECORDED DURING BENCHMARK EXECUTION

Rail	Logged fields	Description
VDD_CORE	_V, _A, _W	Core rail voltage, current, and computed power
3V3_SYS	_V, _A, _W	3.3V system rail voltage, current, and computed power
1V8_SYS	_V, _A, _W	1.8V system rail voltage, current, and computed power
1V1_SYS	_V, _A, _W	1.1V system rail voltage, current, and computed power

10.4.1 Data Structure, Format, and Storage

Experimental results are stored as CSV files, with each file representing one benchmark run under one optimisation configuration. Each row corresponds to a sampled time point, and each column stores either run metadata or measurement values. The dataset includes benchmark identifiers, optimisation variant, runtime information, and per-rail voltage, current, and power values.

TABLE 9: CSV FIELDS GENERATED BY THE LOGGING SCRIPT

Field Group	Field	Description
Run metadata	kernel, variant, run_id, run_index	Identifies the benchmark, optimisation configuration, and repetition.
Timing data	time, run_time	Records sample time and benchmark runtime.
System state	cpu_freq, cpu_temp	Records CPU frequency and temperature during execution.
Aggregate power	total_power	Stores total system-level power for the sampled time point.
Core rail readings	VDD_CORE_V, VDD_CORE_A, VDD_CORE_W	Stores core rail voltage, current, and power.
3.3V rail readings	3V3_SYS_V, 3V3_SYS_A, 3V3_SYS_W	Stores 3.3V system rail voltage, current, and power.
1.8V rail readings	1V8_SYS_V, 1V8_SYS_A, 1V8_SYS_W	Stores 1.8V system rail voltage, current, and power.
1.1V rail readings	1V1_SYS_V, 1V1_SYS_A, 1V1_SYS_W	Stores 1.1V system rail voltage, current, and power.

10.4.2 Power and Energy Calculation

The total_power value is calculated by summing the measured rail power values included in the CSV file:

$$P_{total} = P_{VDD\_CORE} + P_{3v3\_SYS} + P_{1v8\_SYS} + P_{1v1\_SYS}$$

The individual rail power values are calculated from the corresponding voltage and current readings:

$$Power\ (W) = Voltage\ (V) \times Current\ (A)$$

The CSV file does not directly store final energy values. Instead, it stores the power and runtime values needed to calculate energy during analysis. This calculation provides an estimated energy value based on mean sampled power and runtime, rather than a sample-by-sample integration of the power trace.

$$Energy\ (J) = Average\ total\_power\ (W) \times Runtime\ (s)$$

10.5 Design Alternatives Considered

Several design alternatives were considered before the final framework was chosen. At first, a custom C99 loop-based benchmark was considered because it would give direct control over the workload. However, this was replaced with selected PolyBench kernels because one custom program would be too narrow and would be harder to compare with existing compiler research.

Manual benchmark execution was also considered. This was rejected because running commands manually could lead to inconsistent timings, command errors, or missing records. Instead, a controller script was used to automate the execution process and apply the same steps to each benchmark configuration.

External power measurement hardware was another possible option, but it was outside the practical scope of the project. The final design uses Raspberry Pi 5 PMIC readings through the Raspberry Pi power-monitoring interface. This limits the results to system-level power measurement but keeps the experiment practical and repeatable.

11 Implementation

The implementation involved developing automation scripts to compile benchmark binaries, control experiment execution, and record PMIC power readings to CSV files. These scripts produced the measurement dataset and the generated assembly files used for analysis.

11.1 Build System

The first stage of the implementation was the build script, build_binaries.sh. This script was responsible for compiling the selected PolyBench/C kernels into separate binaries for each optimisation configuration.

The build script begins by defining the main project paths used during compilation. These include the base project directory, the PolyBench source directory, the output directory for generated binaries, and a build manifest file used to record the binaries created during compilation. The script also creates separate output folders for each selected benchmark so that the compiled binaries are organised by kernel.

```
BASE="$HOME/dev/power_project"
POLY="$BASE/PolyBenchC-4.2.1"
OUT="$BASE/binaries"
LOG="$BASE/binaries/build_manifest.txt"

mkdir -p "$OUT/gemm" "$OUT/2mm" "$OUT/3mm" "$OUT/corr"
rm -f "$LOG"

UTIL="$POLY/utilities/polybench.c"
COMMON="-mcpu=cortex-a72 -DPOLYBENCH_TIME -DEXTRALARGE_DATASET -I
$POLY/utilities"
```

LISTING 1 : BUILD SCRIPT CONFIGURATION AND SHARED COMPILER OPTIONS.

Listing 1 shows the initial configuration used by the build script. The BASE variable stores the root project directory, while POLY stores the location of the PolyBench/C source code. The OUT variable defines where the compiled binaries are saved, and LOG defines the build manifest file. The build manifest records the binaries created during compilation, making it easier to check that each benchmark and optimisation variant was generated successfully. The mkdir -p command creates separate output directories for each selected benchmark kernel.

The UTIL variable points to the PolyBench utility source file, which is compiled alongside each benchmark. The COMMON variable stores the compiler options shared by all builds. These options set the target processor, enable PolyBench timing, select the EXTRALARGE_DATASET size, and include the PolyBench utility headers. Keeping these options in a shared variable reduces repetition and ensures that each benchmark is compiled using the same baseline settings.

11.1.1 Compile Function

After setting up the project paths, the script defines a reusable compile() function. This function contains the repeated compilation logic used for each benchmark. Instead of writing a separate build process for every PolyBench kernel, the same function is called with different arguments for each of the selected benchmarks.

```
compile () {
    SRC=$1
    NAME=$2
    DIR=$3

    echo "Compiling $NAME..."
    echo "=== $NAME ===" >> "$LOG"

    clang -O0 $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O0"
    echo "O0: ${NAME}_O0" >> "$LOG"

    clang -O2 $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O2"
    echo "O2: ${NAME}_O2" >> "$LOG"

    clang -O3 $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O3"
    echo "O3: ${NAME}_O3" >> "$LOG"

    clang -O3 -fno-vectorize -fno-slp-vectorize $COMMON "$UTIL" "$SRC" -
lm \
        -o "$OUT/$DIR/${NAME}_O3_no_vec"
    echo "O3_no_vec: ${NAME}_O3_no_vec" >> "$LOG"
}
```

LISTING 2: REUSABLE COMPILATION FUNCTION USED TO GENERATE BENCHMARK BINARIES.

Listing 2 shows the reusable compilation function. The function receives three arguments: SRC, which stores the benchmark source file path; NAME, which stores the benchmark name used in the output filename; and DIR, which stores the output folder for the compiled binary. These arguments allow the same function to compile different PolyBench kernels without duplicating the full compilation process.

The function invokes Clang multiple times with different optimisation flags. In the final experiment, the selected optimisation variants were O0, O2, O3, and O3_no_vec. The O3_no_vec variant disables vectorisation using -fno-vectorize and -fno-slp-vectorize, allowing the experiment to compare standard O3 optimisation against an O3 build where vectorisation was disabled. Each compiled binary is saved using a filename that combines the benchmark name and optimisation variant, such as gemm_O2.

```
compile "$POLY/linear-algebra/blas/gemm/gemm.c" "gemm" "gemm"
compile "$POLY/linear-algebra/kernels/2mm/2mm.c" "2mm" "2mm"
compile "$POLY/linear-algebra/kernels/3mm/3mm.c" "3mm" "3mm"
compile "$POLY/datamining/correlation/correlation.c" "corr" "corr"
```

LISTING 3: FUNCTION CALLS USED TO COMPILE THE SELECTED POLYBENCH KERNELS.

Listing 3 shows the function calls used to compile the selected PolyBench kernels. Each call passes the source file path, benchmark name, and output directory into the compile() function. This produced separate binaries for gemm, 2mm, 3mm, and corr, allowing the controller script to locate and execute each benchmark during the experiment.

```
hakeem@hakeem:~/dev/power_project/binaries $ ls
2mm  3mm  build_manifest.txt  corr  gemm  old
hakeem@hakeem:~/dev/power_project/binaries $ cd gemm
hakeem@hakeem:~/dev/power_project/binaries/gemm $ ls
gemm_O0  gemm_O2  gemm_O3  gemm_O3_no_unroll  gemm_O3_no_vec  gemm_O3_unroll
```

LISTING 4: DIRECTORY STRUCTURE SHOWING GENERATED BENCHMARK BINARIES

This build process ensured that all benchmark binaries were produced under the same baseline conditions, with only the optimisation configuration changed between variants. This made the generated binaries suitable for controlled comparison during the measurement stage.

11.2 Experimental Controller

The controller script, run_experiments.py, automated the execution of the benchmark experiments. It selected each benchmark kernel, selected each optimisation variant, repeated each configuration four times, called the logging script for a fixed duration, and recorded completed runs in a state file.

The controller begins by defining the experiment settings. These include the project directory, the path to the logging script, the benchmark kernels, the optimisation variants, the number of repeats, the measurement duration, and the cooldown period between runs. These values are defined near the start of the script so that the experimental configuration can be changed without modifying the main execution logic.

```
BASE_DIR = "/home/hakeem/dev/power_project"
LOGGER = os.path.join(BASE_DIR, "scripts/log_power_final.py")

KERNELS = ["gemm", "2mm", "3mm", "corr"]
VARIANTS = ["O0", "O2", "O3", "O3_no_vec"]

REPEATS = 4
```

```
DURATION = 300
COOLDOWN = 120

STATE_FILE = os.path.join(BASE_DIR, "experiment_state.json")
STOP_FILE = os.path.join(BASE_DIR, "STOP")
```

LISTING 5: CONTROLLER CONFIGURATION AND EXPERIMENT PARAMETERS.

The KERNELS list stores the four PolyBench kernels used in the experiment. The VARIANTS list stores the optimisation levels tested for each kernel. Each benchmark and optimisation pair was repeated four times. Each run lasted 300 seconds, and a 120-second cooldown period was used between runs to reduce the effect of one test carrying over into the next.

The controller also uses two files to make the experiment easier to manage. The STATE_FILE records which runs have already finished, so the script can continue from the last completed run if it is stopped. The STOP_FILE provides a simple way to stop the experiment safely. If the file is found, the controller exits before starting another run.

Before the controller starts the main loop, it checks whether a previous state file already exists. If it does, the script loads the completed runs from the file. If it does not, the script starts with no completed runs recorded.

```
if os.path.exists(STATE_FILE):
    with open(STATE_FILE, "r") as f:
        completed = set(tuple(x) for x in json.load(f))
else:
    completed = set()


def save():
    with open(STATE_FILE, "w") as f:
        json.dump(list(completed), f)
```

LISTING 6: STATE TRACKING USED TO RESUME INTERRUPTED EXPERIMENTS.

The state tracking logic allows the controller to continue from the last completed configuration rather than restarting the full experiment. If the state file exists, the saved run records are loaded into the completed set. If it does not exist, the controller starts with an empty set. After each successful run, the save() function writes the updated set of completed configurations back to disk.

The main execution loop then generates each benchmark run by iterating through the selected kernels, optimisation variants, and repeat index. For each generated configuration, the controller checks whether the run has already been completed before calling the logging script.

```
for kernel in KERNELS:
    for variant in VARIANTS:
        for run in range(REPEATS):

            if os.path.exists(STOP_FILE):
                print("[CTRL][STOP] Detected STOP file. Exiting
cleanly.")
                exit(0)

            key = (kernel, variant, run)

            if key in completed:
                print("[CTRL][SKIP]", key)
                continue

            print(f"\n[CTRL][RUN START] {key}\n")

            subprocess.run([
                "python3",
                LOGGER,
                kernel,
                variant,
                str(DURATION)
            ], check=True)

            completed.add(key)
            save()

            print(f"[CTRL][COOLDOWN] {COOLDOWN}s\n")
            time.sleep(COOLDOWN)

        upload_logs()
```

LISTING 7: MAIN CONTROLLER LOOP FOR AUTOMATED BENCHMARK EXECUTION.

The nested loop structure ensures that every selected benchmark and optimisation variant is executed in a consistent order. The outer loop iterates through the benchmark kernels, the second loop iterates through the optimisation variants, and the inner loop repeats each configuration four times. Since the code uses range(REPEATS), the repeat indices are stored as 0, 1, 2, and 3.

Each run is identified using the tuple key, which contains the kernel name, optimisation variant, and repeat index. If the key is already present in the completed set, the run is skipped. Otherwise, the controller starts the logging script using subprocess.run(), passing the kernel name, optimisation variant, and measurement duration as command-line arguments.

The check=True argument prevents the controller from silently continuing after a failed logging run. If the logging script fails, the controller treats this as an error instead of recording the run as complete. After a successful run, the key is added to the completed set, the state file is updated, and the controller waits for the cooldown period before starting the next run.

During implementation, an additional upload step was added after each optimisation variant had completed its repeated runs. The generated CSV logs were uploaded to Google Drive using rclone, so that they could be accessed from Google Colab for analysis. This was not part of the original core measurement design, but it improved the workflow by separating data collection on the Raspberry Pi from data processing in Colab. This was useful because the Raspberry Pi 5 was used mainly as the measurement device, while Colab provided a more convenient environment for combining CSV files, producing tables, and generating graphs.

Overall, the controller script automated the full experiment sequence by selecting benchmarks, applying optimisation variants, repeating runs, recording completed executions, applying cooldown periods, and backing up output data. This reduced manual intervention and helped make the execution process consistent across the full experiment.

11.3 Logging Script

The logging script is responsible for running one selected benchmark binary at a time and recording the power, timing, and system state data during execution. It is called by the controller script and receives three command line values:

- The Benchmark Kernel
- The Optimisation Variant
- The Measurement Duration

The script then defines the main file locations used during logging. BASE_DIR stores the root project directory, BINARY_DIR points to the compiled benchmark binaries, and LOG_DIR defines where the generated CSV files are saved. These paths allow the logger to find the correct executable and store the output data in a consistent location.

```
BASE_DIR = "/home/hakeem/dev/power_project"
BINARY_DIR = os.path.join(BASE_DIR, "binaries")
LOG_DIR = os.path.join(BASE_DIR, "logs")

INTERVAL = 0.5
PRERUN_TIME = 10

IMPORTANT_RAILS = ["VDD_CORE", "3V3_SYS", "1V8_SYS", "1V1_SYS"]
```

LISTING 8: LOGGER SETTINGS AND SELECTED PMIC RAILS.

The logger records one sample every 0.5 seconds using the INTERVAL value. The PRERUN_TIME value gives the benchmark 10 seconds to begin running before measurements are recorded, which helps reduce the effect of start-up behaviour on the logged data. The selected PMIC rails are stored in IMPORTANT_RAILS. For each rail, the script records voltage, current, and calculated power. These values are then combined to produce the total_power value written to the CSV file.

PMIC readings are collected using vcgencmd pmic_read_adc. The command output provides voltage and current readings for the Raspberry Pi power rails used in the experiment. The script separates these readings into voltage and current dictionaries, so that each rail's voltage can be matched with its corresponding current before power is calculated. The logger also records CPU frequency and CPU temperature, giving extra context for interpreting the power measurements.

```
for rail in IMPORTANT_RAILS:
    v = voltages.get(rail, 0.0)
    i = currents.get(rail, 0.0)
    p = v * i

    row[f"{rail}_V"] = v
    row[f"{rail}_A"] = i
    row[f"{rail}_W"] = p

    total += p
```

LISTING 9: PMIC RAIL POWER CALCULATION

The code in listing 9 calculates the power for each selected rail. For each rail, the script retrieves the voltage and current values and multiplies them together. The voltage, current, and power values are then stored in the output row using a consistent column format. For example, the VDD_CORE rail creates the columns VDD_CORE_V, VDD_CORE_A, and VDD_CORE_W.

The calculated power for each rail is added to the total value. This produces one total power reading for each sample. Storing both the individual rail values and the total power value made the CSV output easier to check during analysis.

The main logging process is handled by the run_experiment() function. This function receives the benchmark name, optimisation variant, and measurement duration from the controller script. These values are used to locate the correct compiled binary inside the binaries directory.

```
def run_experiment(kernel, variant, duration):

    binary_path = os.path.join(BINARY_DIR, kernel,
f"{kernel}_{variant}")

    if not os.path.exists(binary_path):
        print(f"[ERROR] Missing binary: {binary_path}")
        sys.exit(1)

    os.makedirs(LOG_DIR, exist_ok=True)

    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    csv_path = os.path.join(LOG_DIR,
f"{kernel}_{variant}_{timestamp}.csv")
```

LISTING 10: BINARY SELECTION AND CSV FILE CREATION.

Listing 10 shows how the logger selects the correct benchmark binary and creates the CSV output file. The binary path is built using the benchmark name and optimisation variant. For example, if the controller passes gemm and O2, the logger expects to find the binary at binaries/gemm/gemm_O2.

If the binary does not exist, the script prints an error and stops. This prevents the logger from recording invalid data for a missing benchmark. The script also creates the log directory if it does not already exist. A timestamp is added to the CSV filename so that each run creates a separate output file and does not overwrite previous results.

Before the benchmark starts, the script defines the CSV column names. These include the benchmark name, optimisation variant, run identifier, timing values, total power, CPU frequency, CPU temperature, and the voltage, current, and power values for each selected PMIC rail. The PMIC rail columns are generated from the IMPORTANT_RAILS list, which keeps the CSV format consistent with the selected rails.

The main measurement loop starts the selected binary and continues until the requested duration has passed. The benchmark is started using subprocess.Popen(), which allows the logger to keep collecting measurements while the benchmark is running.

```
proc = subprocess.Popen([binary_path])
run_start = time.time()

time.sleep(PRERUN_TIME)

while True:

    now = time.time()
    t = now - start_global

    if t > duration:
        break

    if proc.poll() is not None:
        proc.wait()
        run_index += 1
        run_id = str(uuid.uuid4())

        proc = subprocess.Popen([binary_path])
        run_start = time.time()

    run_time = now - run_start

    freq = read_cpu()
    temp = read_temp()

    pmic, total = read_pmic()
```

LISTING 11: MAIN LOGGING LOOP USED DURING MEASUREMENT.

The loop first checks how much time has passed since the start of the measurement. If the selected duration has been reached, the loop ends. Otherwise, the logger checks whether the benchmark process is still running.

Some PolyBench benchmarks can finish before the full measurement duration has passed. When this happens, the logger restarts the benchmark so that the workload remains active for the full run. Each restarted execution receives a new run_id, while run_index records how many benchmark executions have occurred during the measurement window.

During each loop, the logger records CPU frequency, CPU temperature, and PMIC readings. These values are then combined into one CSV row.

```
row = {
    "kernel": kernel,
    "variant": variant,
    "run_id": run_id,
    "run_index": run_index,
    "time": round(t, 3),
    "run_time": round(run_time, 3),
    "total_power": total,
    "cpu_freq": freq,
    "cpu_temp": temp
}

row.update(pmic)
writer.writerow(row)
```

LISTING 12: CSV ROW WRITTEN FOR EACH MEASUREMENT SAMPLE.

Each row contains the benchmark name, optimisation variant, run identifier, execution index, timing values, total power, CPU frequency, and CPU temperature. The PMIC rail readings are added using row.update(pmic), which adds the voltage, current, and power values for each selected rail. This creates one complete CSV row for each sample.

At the end of the measurement period, the benchmark process is stopped and the CSV file path is printed. Overall, the logging script selects the correct binary, keeps the workload active during the measurement period, records power and system state values at fixed intervals, and saves the results in a structured CSV file for analysis.

```
hakeem@hakeem:~/dev/power_project $ ls
binaries  experiment_state.json  logs  PolyBenchC-4.2.1  scripts
hakeem@hakeem:~/dev/power_project $
```

FIGURE 3: FINAL PROJECT STRUCTURE AFTER IMPLEMENTATION

Figure 3 shows the final project directory after the implementation stage. The directory contains the main components of the artefact, including the benchmark binaries, scripts, logs, PolyBench source files, and experiment state file.

12 Testing and Demonstration

The framework was tested in stages to confirm that each part of the experiment worked before running the full measurement process. The testing focused on three areas: whether the build script produced the correct binaries, whether the controller executed the correct benchmark configurations, and whether the logging script produced usable CSV output.

12.1 Build Script Test

The first test checked that the build script compiled each selected PolyBench kernel under the required optimisation configurations. After running the build script, the output directory was inspected to confirm that separate binary files had been created for each kernel and optimisation variant.

```
hakeem@hakeem:~/dev/power_project/scripts $ ./build_binaries.sh
Building PolyBench LLVM binaries (EXTRALARGE_DATASET)...
Compiling gemm...
Compiling 2mm...
Compiling 3mm...
Compiling corr...
Build complete.
```

FIGURE 4: TERMINAL OUTPUT SHOWING SUCCESSFUL EXECUTION OF THE BUILD SCRIPT

```
hakeem@hakeem:~/dev/power_project/binaries $ ls
2mm  3mm  build_manifest.txt  corr  gemm
```

FIGURE 5: DIRECTORY STRUCTURE SHOWING GENERATED BENCHMARK BINARIES

The output confirmed that the build script completed successfully. For example, the gemm folder contained binaries such as gemm_O0, gemm_O2, gemm_O3, and gemm_O3_no_vec. This confirmed that the controller would be able to find the correct binary during the experiment.

12.2 Controller Script Test

The controller script was then tested to confirm that it selected the correct kernel and optimisation variant, then correctly called the logging script and moved through the planned execution sequence. A shorter test duration was used before the full experiment so that the workflow could be checked without waiting for the complete measurement period.

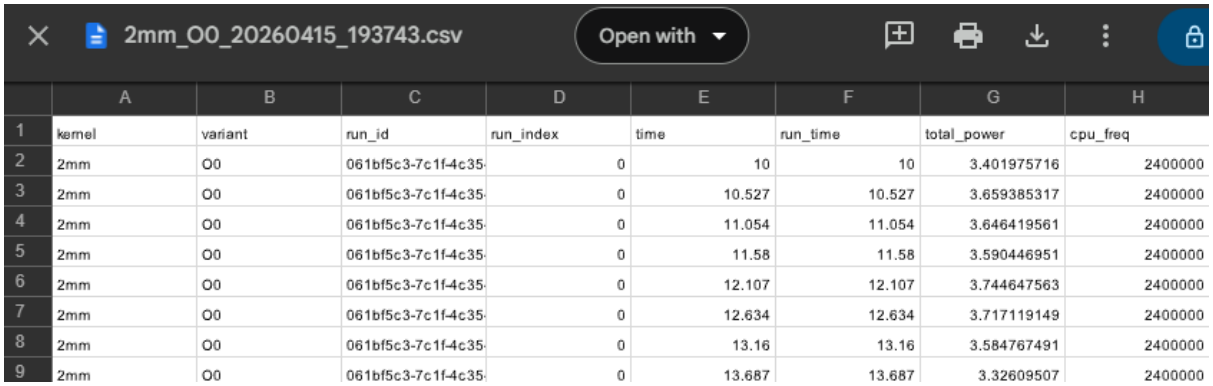
```
gemm O0 | exec_run=0 | t=49.0s | run_t=48.98s | P=3.88W | T=53.2C
gemm O0 | exec_run=0 | t=49.5s | run_t=49.51s | P=4.07W | T=52.7C
gemm O0 | exec_run=0 | t=50.0s | run_t=50.03s | P=3.88W | T=52.7C
gemm O0 | exec_run=0 | t=50.6s | run_t=50.56s | P=3.94W | T=52.7C
gemm O0 | exec_run=0 | t=51.1s | run_t=51.09s | P=3.89W | T=52.7C
gemm O0 | exec_run=0 | t=51.6s | run_t=51.61s | P=3.87W | T=53.2C
51.565753
[RUN COMPLETE] exec_run=0 duration=52.14s
[RESTART] exec_run=1
gemm O0 | exec_run=1 | t=52.1s | run_t=-0.00s | P=3.42W | T=48.8C
gemm O0 | exec_run=1 | t=52.7s | run_t=0.53s | P=3.94W | T=49.9C
gemm O0 | exec_run=1 | t=53.2s | run_t=1.05s | P=3.97W | T=49.9C
```

FIGURE 6: CONTROLLER TERMINAL OUTPUT SHOWING BENCHMARK EXECUTION AND COOLDOWN MESSAGES.

This test confirmed that the controller could pass the kernel name, optimisation variant, and duration to the logging script correctly. It also confirmed that completed runs were recorded in the state file, allowing the experiment to resume safely if interrupted.

12.3 Logging Script Test

The logging script was tested by running one benchmark configuration and checking the CSV file that was produced. The output file was inspected to confirm that it contained the expected metadata, timing values, CPU information, total power values, and PMIC rail readings.



	A	B	C	D	E	F	G	H
1	kernel	variant	run_id	run_index	time	run_time	total_power	cpu_freq
2	2mm	O0	061bf5c3-7c1f-4c35-	0	10	10	3.401975716	2400000
3	2mm	O0	061bf5c3-7c1f-4c35-	0	10.527	10.527	3.659385317	2400000
4	2mm	O0	061bf5c3-7c1f-4c35-	0	11.054	11.054	3.646419561	2400000
5	2mm	O0	061bf5c3-7c1f-4c35-	0	11.58	11.58	3.590446951	2400000
6	2mm	O0	061bf5c3-7c1f-4c35-	0	12.107	12.107	3.744647563	2400000
7	2mm	O0	061bf5c3-7c1f-4c35-	0	12.634	12.634	3.717119149	2400000
8	2mm	O0	061bf5c3-7c1f-4c35-	0	13.16	13.16	3.584767491	2400000
9	2mm	O0	061bf5c3-7c1f-4c35-	0	13.687	13.687	3.32609507	2400000

FIGURE 7: EXAMPLE CSV OUTPUT GENERATED BY THE LOGGING SCRIPT.

This confirmed that the logger was recording usable measurement data. The CSV structure also made it possible to link each row of data back to the benchmark kernel, optimisation variant, and execution run.

12.4 Full Workflow Test

After the individual scripts had been tested, the full workflow was tested by running the controller script with the build and logging components in place. This demonstrated that the complete framework could compile binaries, execute the benchmark configurations, collect power readings, and store the results for later analysis.

```
hakeem@hakeem:~/dev/power_project/logs $ ls
2mm_O0_20260415_193042.csv  2mm_O3_20260415_204752.csv         3mm_O2_20260415_220505.csv         corr_O0_20260415_232252.csv
2mm_O0_20260415_193743.csv  2mm_O3_no_vec_20260415_205456.csv  3mm_O2_20260415_221205.csv         corr_O0_20260415_232952.csv
2mm_O0_20260415_194443.csv  2mm_O3_no_vec_20260415_210156.csv  3mm_O3_20260415_221942.csv         corr_O0_20260415_233653.csv
2mm_O0_20260415_195144.csv  2mm_O3_no_vec_20260415_210856.csv  3mm_O3_20260415_222643.csv         corr_O2_20260415_234356.csv
2mm_O2_20260415_195847.csv  2mm_O3_no_vec_20260415_211557.csv  3mm_O3_20260415_223343.csv         corr_O2_20260415_235056.csv
2mm_O2_20260415_200547.csv  3mm_O0_20260415_212300.csv         3mm_O3_20260415_224044.csv         corr_O2_20260415_235757.csv
2mm_O2_20260415_201248.csv  3mm_O0_20260415_213000.csv         3mm_O3_no_vec_20260415_224747.csv  corr_O2_20260416_000457.csv
2mm_O2_20260415_201948.csv  3mm_O0_20260415_213701.csv         3mm_O3_no_vec_20260415_225448.csv  corr_O3_20260416_001200.csv
2mm_O3_20260415_202652.csv  3mm_O0_20260415_214401.csv         3mm_O3_no_vec_20260415_230148.csv  corr_O3_20260416_001900.csv
2mm_O3_20260415_203352.csv  3mm_O2_20260415_215104.csv         3mm_O3_no_vec_20260415_230848.csv  corr_O3_20260416_002600.csv
2mm_O3_20260415_204052.csv  3mm_O2_20260415_215805.csv         corr_O0_20260415_231552.csv        corr_O3_20260416_003300.csv
hakeem@hakeem:~/dev/power_project/logs $
```

FIGURE 8: OVERALL PROJECT FOLDER SHOWING SCRIPTS, BINARIES, LOGS, AND CSV OUTPUT

The full workflow test confirmed that the artefact worked as an end-to-end experimental system rather than as separate scripts. The build script produced the benchmark binaries, the controller managed the execution sequence, and the logging script recorded the measurement data in a structured format.

TABLE 10: SUMMARY OF IMPLEMENTATION TESTING

Test Area	Expected Result	Evidence	Outcome
Build script	Binaries are created for each selected kernel and optimisation variant	Terminal output and binary folders	Passed
Controller script	Correct kernel, variant, and duration are passed to the logging script	Controller terminal output	Passed
Logging script	CSV file contains metadata, timing, CPU, total power, and rail readings	CSV output screenshot	Passed
Full workflow	Scripts work together as one experiment pipeline	Project folder and generated logs	Passed

After testing was completed, the full experiment was executed using the selected PolyBench kernels and LLVM optimisation configurations. The complete run took approximately seven hours, including measurement periods and cooldown delays between runs. This produced 69 CSV output files, including the final measurement logs used for analysis in the next section.

13 Analysis and Findings

This section analyses the CSV logs produced by the experimental framework. The analysis compares runtime, average power, and estimated energy consumption across the selected PolyBench kernels and LLVM optimisation configurations.

13.1 Data Preparation

The experiment produced multiple CSV log files, with each file representing one benchmark and optimisation configuration run. These files were uploaded into Google Colab and processed using Python. Colab was used as the analysis environment because it allowed the CSV files to be combined, checked, grouped, and visualised in one place without changing the original experimental data.

```
import pandas as pd
import glob
import os

files = glob.glob("/content/logs/*.csv")
rows = []

for file in files:
    df = pd.read_csv(file)

    kernel = df["kernel"].iloc[0]
    variant = df["variant"].iloc[0]
    runtime = df["time"].max()
    avg_power = df["total_power"].mean()
    energy = avg_power * runtime

    rows.append({
        "kernel": kernel,
        "variant": variant,
        "runtime_s": runtime,
        "avg_power_W": avg_power,
        "energy_J": energy,
        "source_file": os.path.basename(file)
    })

summary = pd.DataFrame(rows)
grouped = summary.groupby(["kernel", "variant"]).agg(
    mean_runtime_s=("runtime_s", "mean"),
    mean_power_W=("avg_power_W", "mean"),
    mean_energy_J=("energy_J", "mean"),
    std_energy_J=("energy_J", "std"),
    runs=("energy_J", "count")
).reset_index()
```

LISTING 13: GOOGLE COLAB PYTHON CODE USED TO COMBINE CSV FILES AND CALCULATE SUMMARY METRICS

The code in Listing 13 was used to process the CSV logs generated during the experiment. Each CSV file was read into Google Colab, and the main values needed for analysis were extracted. These included the benchmark name, optimisation variant, runtime, average total power, and estimated energy.

Energy was calculated using the method defined system design section:

$$Energy(J) = Average\ total\_power(W)\ x\ Runtime(s)$$

The combined dataset was then grouped by benchmark kernel and optimisation variant. This allowed repeated runs of the same configuration to be averaged, so that runtime, power, and energy could be compared consistently across the optimisation configurations.

TABLE 11: SUMMARY METRICS CALCULATED FROM THE CSV LOGS

Metric	**Description**
mean_runtime_s	Average runtime for each benchmark and optimisation variant
mean_power_W	Average sampled system power during execution
mean_energy_J	Estimated average energy consumption
std_energy_J	Variation in energy across repeated runs
runs	Number of CSV files included for that configuration

The grouped dataset in Table 12 forms the basis for the analysis in the following sections. Runtime, power, and energy are examined separately first, before comparing the overall energy-efficiency effects of the optimisation configurations.

13.2 Runtime Comparison

The first comparison examined benchmark runtime. Runtime was included because compiler optimisations often aim to reduce execution time, and shorter execution time can reduce energy consumption if power does not increase too much.

The runtime results are presented with both the mean execution time and the standard deviation across repeated executions. This makes it possible to compare performance while also checking how much variation occurred between runs.

TABLE 12: MEAN BENCHMARK EXECUTION TIME BY KERNEL AND OPTIMISATION VARIANT

Kernel	Variant	Mean runtime(s)	Standard deviation(s)	Executions recorded
2mm	O0	149.61	48.20	8
2mm	O2	59.52	4.94	20
2mm	O3	56.65	13.17	21
2mm	O3_no_vec	59.52	7.94	20
3mm	O0	149.58	151.88	8
3mm	O2	74.54	18.86	16
3mm	O3	74.58	23.63	16
3mm	O3_no_vec	70.15	21.85	17
corr	O0	149.59	24.94	8
corr	O2	45.66	12.92	26
corr	O3	47.50	9.57	25
corr	O3_no_vec	49.49	3.89	24
gemm	O0	49.51	5.89	24
gemm	O2	6.73	0.91	160
gemm	O3	6.72	0.91	160
gemm	O3_no_vec	12.53	0.47	92

The “executions recorded” column refers to benchmark executions captured within the CSV logs, not the number of CSV files, because the logger restarted short-running benchmarks during the fixed measurement window.

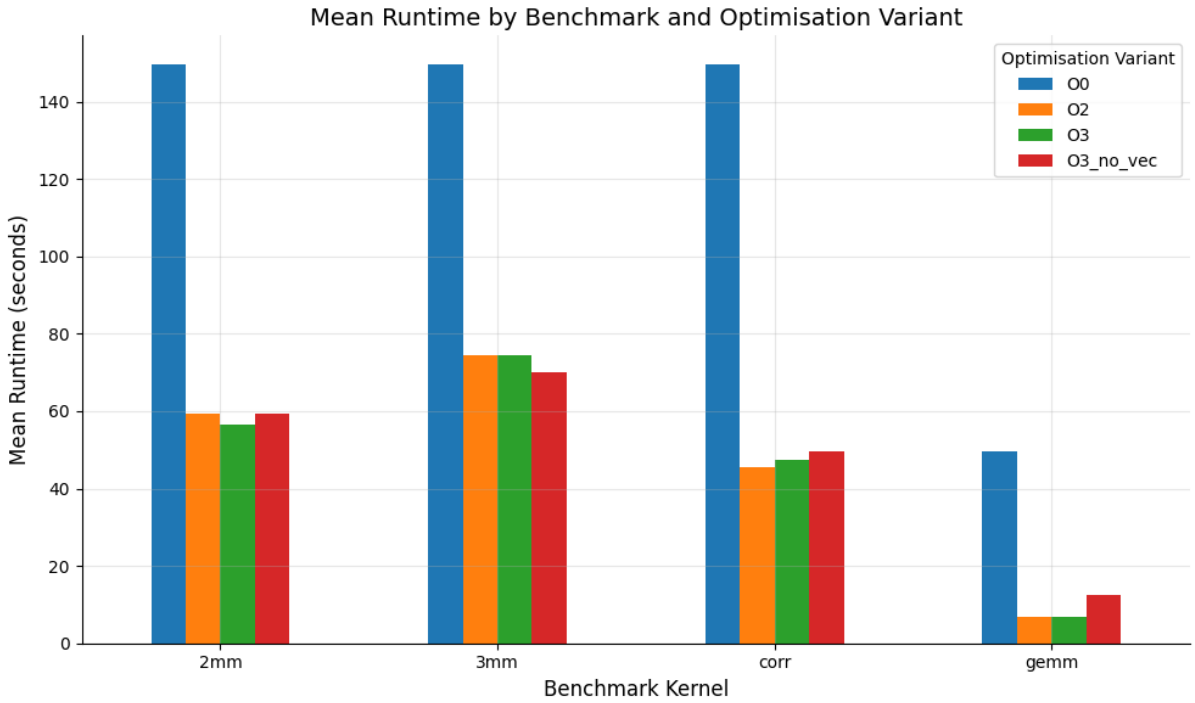


FIGURE 9: MEAN RUNTIME COMPARISON ACROSS OPTIMISATION VARIANTS

The results show that the unoptimised O0 configuration generally produced the longest runtimes. This is most visible for 2mm, 3mm, and corr, where the mean runtime under O0 was much higher than the optimised variants. For example, 2mm decreased from around 149.61 seconds under O0 to around 56.65 seconds under O3. This suggests that LLVM optimisation had a clear performance effect on these loop-intensive workloads.

The optimised variants were much closer to each other. For 2mm, O2, O3, and O3_no_vec all produced runtimes around 56–60 seconds. For 3mm, the fastest mean runtime was seen under O3_no_vec, while O2 and O3 were very similar. For gemm, O2 and O3 produced almost identical runtimes at around 6.72 seconds, while O3_no_vec was slower at around 12.53 seconds. This suggests that vectorisation had a stronger performance effect on gemm than on some of the other kernels.

The standard deviation column is useful because it shows how consistent the repeated runs were. A larger value means the runtime varied more between executions. For example, 3mm under O0 had a high standard deviation, which suggests less stable runtime behaviour. In contrast, gemm under O2 and O3 had much lower variation, so those results appear more consistent.

This runtime comparison is important for the next stage of analysis because lower runtime does not automatically mean lower energy consumption. Energy also depends on the average power drawn during execution, so the power and energy results need to be compared before deciding which optimisation was most energy efficient.

13.3 Average Power Comparison

The next stage of analysis compared the average power consumption for each benchmark and optimisation variant. This was necessary because execution time alone does not fully explain energy behaviour. An optimisation may reduce runtime, but it may also increase the amount of power drawn while the program is running.

The average power values were calculated from the **total_power** column in each CSV file. For each benchmark and optimisation variant, the repeated runs were grouped together to calculate the mean power, standard deviation, and number of recorded executions.

TABLE 13: MEAN AVERAGE POWER BY KERNEL AND OPTIMISATIONS VARIANT

Kernel	Variant	Mean power(W)	Standard deviation(W)	Executions recorded
2mm	O0	3.568	0.020	8
2mm	O2	4.771	0.041	20
2mm	O3	4.737	0.131	21
2mm	O3_no_vec	4.770	0.030	20
3mm	O0	3.563	0.041	8
3mm	O2	5.046	0.082	16
3mm	O3	5.058	0.098	16
3mm	O3_no_vec	5.022	0.058	17
corr	O0	3.673	0.024	8
corr	O2	4.364	0.073	26
corr	O3	4.339	0.057	25
corr	O3_no_vec	4.433	0.058	24
gemm	O0	4.104	0.037	24
gemm	O2	5.194	0.078	160
gemm	O3	5.181	0.081	160
gemm	O3_no_vec	4.821	0.055	92

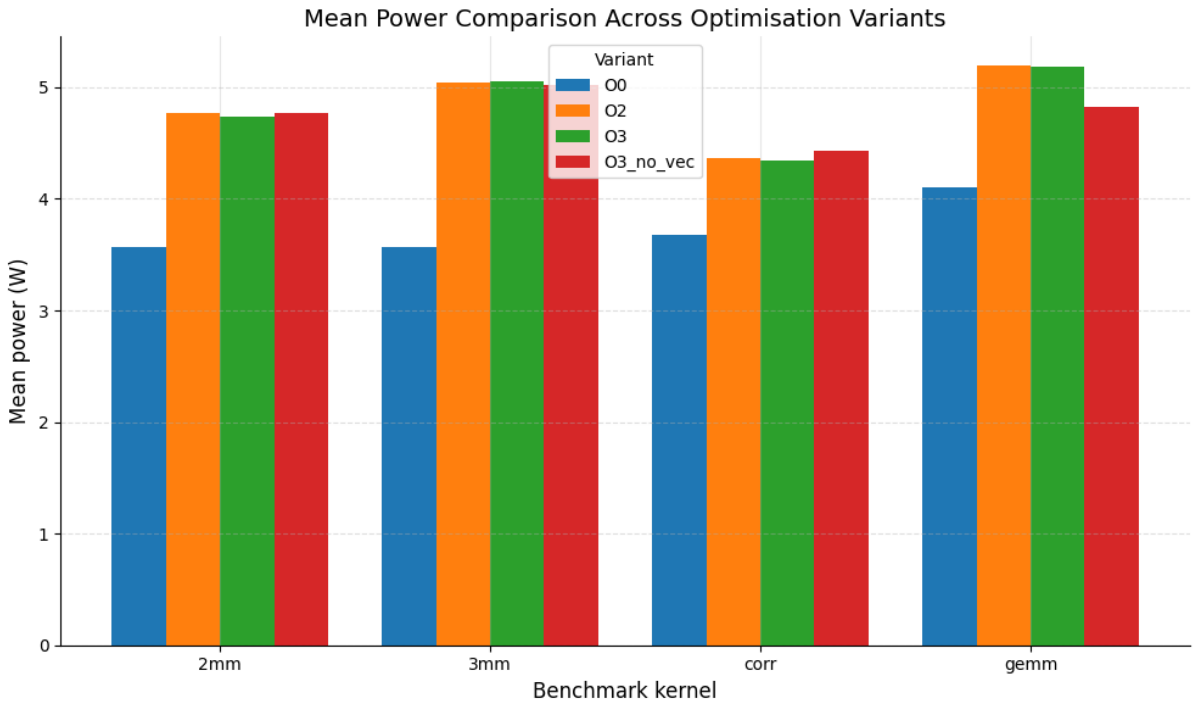


FIGURE 10: MEAN POWER COMPARISON ACROSS OPTIMISATION VARIANTS

The average power results show that the optimised configurations generally used more power than the un-optimised O0 baseline. This pattern appears across all four benchmark kernels. For example, 2mm increased from 3.568 W under O0 to around 4.7 W under the optimised variants. A similar pattern can be seen in 3mm, where the average power increased from 3.563 W under O0 to just over 5 W under O2, O3, and O3_no_vec.

However, this does not mean that the optimised variants are less energy efficient. The runtime results showed that the optimised configurations often completed much faster than O0. Therefore, even though they drew more power while running, they may still consume less total energy overall. This is why power must be interpreted together with runtime rather than on its own.

The standard deviation values are generally small, which suggests that the average power measurements were fairly stable across repeated executions. For example, most standard deviation values are below 0.1 W, although 2mm under O3 shows slightly higher variation at 0.131 W. This supports the reliability of the power comparison, while still showing that some configurations had more variation than others.

A useful finding here is that O3_no_vec does not always reduce power compared with standard O3. For gemm, disabling vectorisation reduced average power from 5.181 W to 4.821 W. However, for corr, O3_no_vec increased average power compared with O3. This suggests that the effect of vectorisation on power depends on the benchmark workload.

Overall, the power comparison shows that compiler optimisation can increase average power draw during execution. However, energy efficiency depends on both power and runtime. The next section therefore combines these values to estimate total energy consumption for each benchmark and optimisation variant.

13.4 Energy Consumption Comparison

Energy consumption was the main comparison in the analysis because it combines both the runtime and power behaviour. A configuration may draw more power while running but still use less total energy if it finishes the benchmark much faster. For this reason, energy provides a clever measure of overall efficiency than runtime or average power alone.

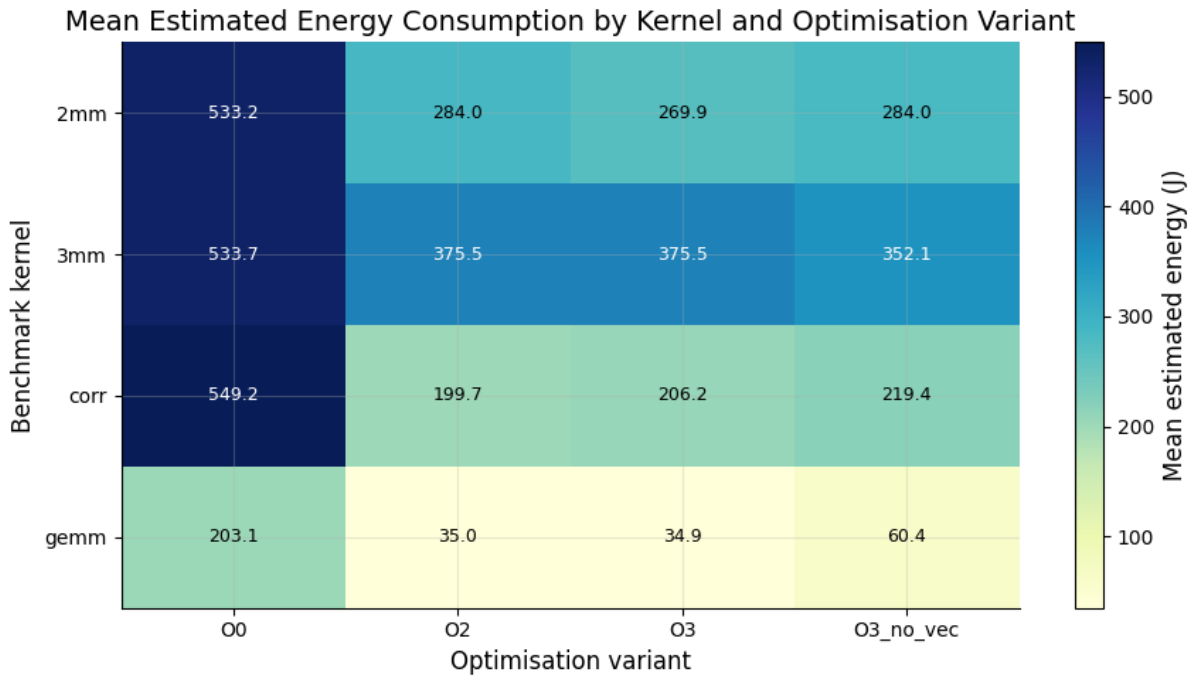


FIGURE 11: HEATMAP SHOWING MEAN ESTIMATED ENERGY CONSUMPTION BY KERNEL AND OPTIMISATION VARIANT

The heatmap gives a direct comparison of energy use across the experiment. Darker or higher-value cells show configurations with greater estimated energy consumption, while lower-value cells indicate more energy-efficient configurations. This makes it easier to identify which optimisation settings reduced total energy use for each benchmark.

TABLE 14: MEAN ESTIMATED ENERGY CONSUMPTION BY KERNEL AND OPTIMISATION VARIANT

Kernel	Variant	Mean energy(j)	Standard deviation(j)	Executions recorded
2mm	O0	533.18	169.65	8
2mm	O2	284.04	24.12	20
2mm	O3	269.86	63.36	21
2mm	O3_no_vec	283.99	38.58	20
3mm	O0	533.66	541.97	8
3mm	O2	375.52	93.85	16
3mm	O3	375.54	116.48	16
3mm	O3_no_vec	352.07	109.43	17
corr	O0	549.17	90.29	8
corr	O2	199.73	56.64	26
corr	O3	206.24	41.82	25
corr	O3_no_vec	219.42	17.92	24
gemm	O0	203.09	23.51	24
gemm	O2	35.00	4.82	160
gemm	O3	34.91	4.81	160
gemm	O3_no_vec	60.41	2.30	92

The results show that O0 produced the highest estimated energy for every benchmark. Across the four O0 configurations, the mean estimated energy was 454.77 J, compared with 224.73 J across the optimised configurations. This represents an average reduction of approximately 50.58%, showing that LLVM optimisation improved estimated energy efficiency overall.

The main reason for this improvement was reduced runtime. The average O0 runtime was 124.57 seconds, while the average runtime across the optimised variants was 46.97 seconds, a reduction of approximately 62.30%. Although average power increased from 3.727 W to 4.811 W, the runtime reduction was large enough to reduce total estimated energy. This is consistent with Georgiou et al. (2018), who found that energy-consumption improvements were closely related to execution-time improvements when evaluating LLVM optimisation configurations on embedded processors.

The most energy-efficient configuration varied by kernel: O3 was best for 2mm and gemm, O2 was best for corr, and O3_no_vec was best for 3mm. This shows that LLVM optimisation reduced estimated energy consumption overall, mainly because runtime savings outweighed the increase in average power draw. However, the results also show that optimisation effects were workload dependent. The findings therefore support the value of compiler optimisation for energy efficiency, but they do not support a simple claim that one optimisation setting is always best.

13.5 Energy Efficiency Relative to O0

To measure the energy benefit of optimisation more clearly, each optimised configuration was compared against the unoptimised O0 baseline for the same benchmark. This shows whether optimisation reduced or increased estimated energy consumption relative to the baseline.

The percentage change was calculated using the mean estimated energy for each variant and the mean estimated energy for O0 on the same benchmark. The formula used was:

$$Energy\ change\ vs\ O0\ (\%)\ =\ ((Optimised\ Energy\ -\ O0\ energy)\ /\ O0\ energy)\ x\ 100$$

A negative value means that the optimised configuration used less estimated energy than O0. A positive value would mean that it used more estimated energy than O0.

TABLE 15: ENERGY CHANGE OF OPTIMISED VARIANTS RELATIVE TO O0

Kernel	Variant	Mean energy (J)	Energy change vs O0%
2mm	O2	284.04	-46.73
2mm	O3	269.86	-49.39
2mm	O3_no_vec	283.99	-46.74
3mm	O2	375.52	-29.63
3mm	O3	375.54	-29.63
3mm	O3_no_vec	352.07	-34.03
corr	O2	199.73	-63.63
corr	O3	206.24	-62.45
corr	O3_no_vec	219.42	-60.04
gemm	O2	35.00	-82.76
gemm	O3	34.91	-82.81
gemm	O3_no_vec	60.41	-70.25

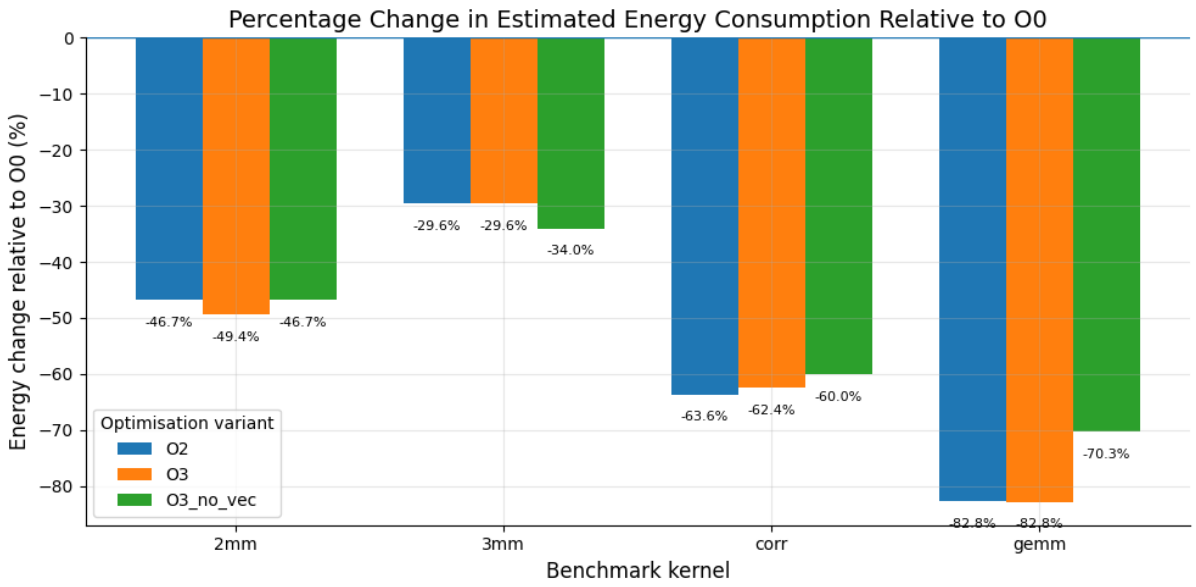


FIGURE 12: PERCENTAGE OF ENERGY CHANGES RELATIVE TO O0

The results show that every optimised configuration reduced estimated energy consumption compared with O0. Across all optimised configurations, the average reduction was approximately 54.8%. The smallest reduction was approximately 29.6%, seen in 3mm under O2 and O3, while the largest reduction was approximately 82.8%, seen in gemm under O3.

The size of the improvement varied between benchmarks. gemm showed the strongest average reduction, with the optimised variants reducing estimated energy by approximately 78.6% compared with O0. This is likely because gemm is a regular matrix multiplication kernel with repeated loop operations over array elements, making it well suited to vectorisation. Maleki et al. (2011) explain that vectorisation targets loop hot spots where the same operation is applied across different array elements, allowing vector instructions to operate on multiple data items simultaneously. This helps explain why gemm benefited strongly from O2 and O3, and why disabling vectorisation under O3_no_vec caused a much higher energy value.

However, 3mm showed the smallest average improvement, at approximately 31.1%. This may be because 3mm involves multiple matrix multiplication stages, increasing memory access and data movement, so the runtime improvement did not translate into as large an energy saving. This supports the view that optimisation benefits are workload dependent rather than uniform across all loop-intensive programs.

Overall, the results show that LLVM optimisation improved estimated energy efficiency for all tested benchmarks. The findings are consistent with the idea that loop-level optimisation can reduce energy consumption by shortening execution time, even when average power draw increases. However, because O2 and O3 apply multiple compiler transformations at the same time, these results should not be interpreted as proof that software pipelining alone caused the improvement. Instead, they provide stronger evidence for a broader conclusion: LLVM's loop-level optimisation behaviour changed runtime and generated-code efficiency in ways that reduced estimated energy consumption for these workloads.

13.6 O3 and O3_no_vec Comparison

The next comparison focused on O3 and O3_no_vec. This comparison is useful because both configurations use aggressive optimisation, but O3_no_vec disables vectorisation using -fno-vectorize and -fno-slp-vectorize. This makes the comparison useful for checking whether the energy behaviour was mainly linked to vectorised code or to wider loop-level optimisation effects.

This matters because the selected benchmarks are loop-intensive. Standard O3 may apply vectorisation, instruction scheduling, loop restructuring, register allocation, and other scheduling behaviour related to software pipelining. O3_no_vec does not isolate software pipelining directly, but it narrows the comparison by removing vectorisation while keeping other aggressive optimisation behaviour.

The percentage difference was calculated using O3 as the baseline:

$$Difference\ (\%) = ((O3\_no\_vec\ energy - O3\ energy) / O3\ energy) \times 100$$

A positive value means that O3_no_vec used more energy than O3. A negative value means that O3_no_vec used less energy than O3.

TABLE 16: ENERGY COMPARISON BETWEEN O3 AND O3_NO_VEC

Kernel	O3 energy (J)	O3_no_vec energy(J)	Difference(%)
2mm	269.86	283.99	5.23
3mm	375.54	352.07	-6.25
corr	206.24	219.42	6.39
gemm	34.91	60.41	73.07

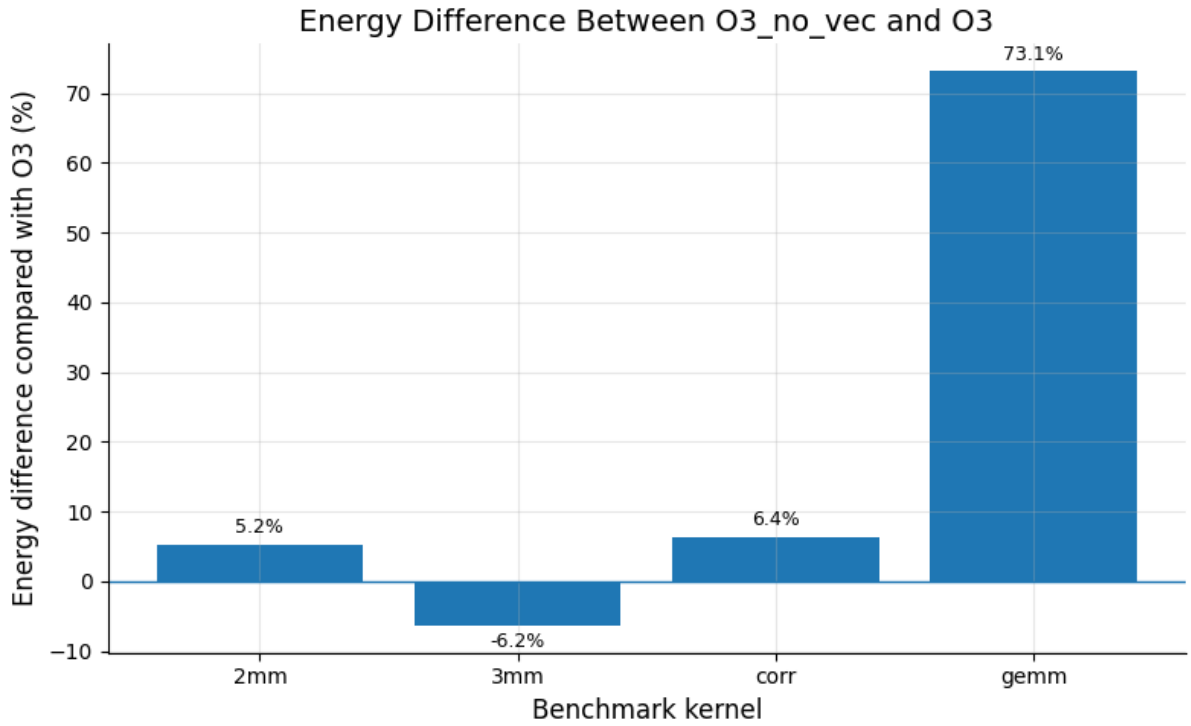


FIGURE 13: PERCENTAGE ENERGY DIFFERENCE BETWEEN O3_NO_VEC AND O3

The results show that disabling vectorisation affected each benchmark differently. For 2mm and corr, O3_no_vec used slightly more estimated energy than O3, with increases of 5.23% and 6.39%. This suggests that standard O3 was more energy efficient for these workloads, but only by a moderate margin.

The strongest difference was seen in gemm, where O3_no_vec used 73.07% more estimated energy than O3. This suggests that the standard O3 build produced much more efficient loop code for this benchmark. Since gemm is a regular matrix multiplication kernel, it was likely able to benefit strongly from vectorisation and related loop-level optimisation.

The 3mm result was different. O3_no_vec used 6.25% less estimated energy than O3, suggesting that vectorisation was not always beneficial for energy efficiency. One possible reason is that 3mm involves multiple matrix multiplication stages, increasing memory access and data movement. In this case, the vectorised O3 code may not have reduced total energy as effectively.

Overall, the comparison shows that aggressive optimisation was workload dependent. O3 was more energy efficient for gemm, corr, and 2mm, while O3_no_vec was slightly more efficient for 3mm. This does not prove that software pipelining alone caused the energy differences, but it does show that changing the generated loop code changed measured energy behaviour. The assembly inspection therefore checks whether these optimisation variants produced visible differences in low-level loop structure, instruction scheduling, and vectorised operations.

14 Assembly Level Inspection

14.1 Purpose of Assembly Inspection

The energy measurements showed that LLVM optimisation changed runtime, average power, and estimated energy consumption. However, these measurements alone do not identify which compiler transformations caused the differences. To provide additional evidence, the generated assembly files were inspected for each benchmark and optimisation variant. The purpose of this inspection was to check whether the optimisation variants produced visibly different loop code, and whether there was evidence of loop scheduling behaviour related to software pipelining.

Assembly output was generated for each selected PolyBench kernel under O0, O2, O3, and O3_no_vec. This allowed the measured results to be compared with the machine-level code produced by each optimisation configuration.

```
clang -S -O3 $COMMON "$UTIL" "$SRC" -lm \
    -o "$ASM_OUT/${NAME}_O3.s"

clang -S -O3 -fno-vectorize -fno-slp-vectorize $COMMON "$UTIL" "$SRC" -
lm \
    -o "$ASM_OUT/${NAME}_O3_no_vec.s"
```

LISTING 14: COMMANDS USED TO GENERATE ASSEMBLY FILES FOR O3 AND O3_NO_VEC

The -S option was used to produce assembly output instead of an executable binary. The same benchmark source files, utility file, dataset size, target processor flag, and optimisation settings were used as in the build stage. This helped ensure that the assembly files reflected the same compiler configurations used during measurement.

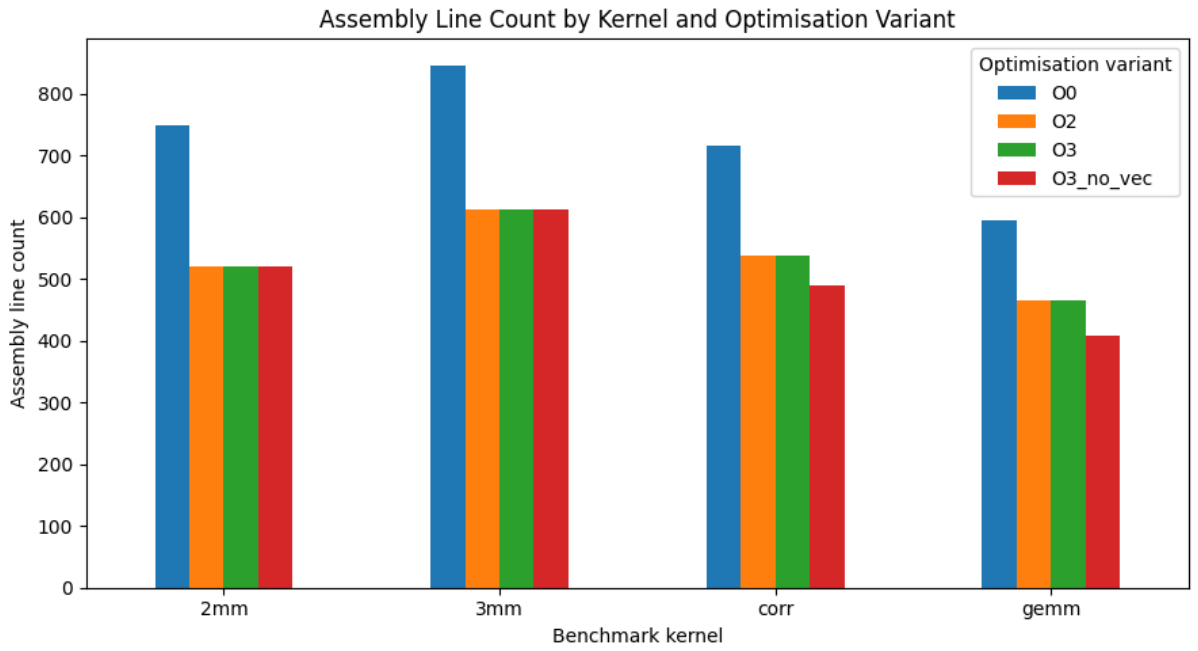


FIGURE 14: ASSEMBLY LINE COUNT BY KERNEL AND OPTIMISATION VARIANT

Figure 14 shows that the O0 assembly files were consistently larger than the optimised versions. On average, the optimised variants reduced assembly line count by approximately 26.5% for O2 and O3, and approximately 30.0% for O3_no_vec, compared with O0. This suggests that LLVM optimisation noticeably changed the generated machine code compared with the unoptimised baseline. This supports the runtime and energy findings because the O0 builds were also generally slower and had higher estimated energy consumption.

However, the optimised variants were much closer to each other in line count. This suggests that the largest structural change came from enabling optimisation, rather than from the smaller differences between O2, O3, and O3_no_vec. For this reason, the line-count comparison is treated as supporting evidence only. Line count shows that the generated code changed, but it does not show what changed inside the loop body.

14.2 Instruction Pattern Comparison

Software pipelining aims to improve loop execution by overlapping work from different loop iterations. Carr, Ding and Sweany explain that "software pipelining can generate efficient schedules for loops by overlapping execution of operations from different iterations" (1995, p.2). This is relevant because the selected PolyBench benchmarks are loop-heavy, so changes inside repeated loop bodies can affect both runtime and energy consumption.

Assembly output cannot prove software pipelining on its own because it shows the final generated code, not the exact LLVM pass responsible for it. However, it can provide supporting evidence. For this reason, selected gemm instruction patterns were counted.

The aim was to check whether LLVM changed the loop from a simple scalar loop into a more efficient loop body. The counted patterns included fused multiply-add instructions, vector-register use, scalar-register use, and branch/control instructions. These were useful because gemm repeatedly performs multiply-add work inside loops, so changes in these instruction types can show whether LLVM made the loop do more useful computation with less loop-control overhead.

TABLE 17: SELECTED ASSEMBLY INSTRUCTION INDICATORS FOR GEMM

Variant	fmla instructions	Fmadd instructions	Vector register instructions	Scalar d-register instructions	Branch/control instructions
O0	0	1	0	32	68
O3	4	2	31	32	23
O3_no_vec	0	2	0	43	16

The table shows a clear difference between O0 and O3. O0 contained no vector-register instructions and only one fused multiply-add instruction, while also having the highest number of branch/control instructions. This suggests that the unoptimised version kept a more direct scalar loop structure, with more loop-control overhead and less evidence of advanced loop scheduling.

In contrast, O3 contained vector-register instructions and fmla operations. The vector-register instructions show that the loop could process multiple values at once, while the fmla instructions show that multiplication and addition were combined into fewer arithmetic operations. The lower number of branch/control instructions is also in line with this, because it

suggests that less loop overhead was needed for the amount of work being performed. This shows that LLVM generated a more efficient loop body under O3, which supports the energy result where gemm recorded its lowest estimated energy value.

14.3 Assembly Comparison for gemm

The assembly comparison used the gemm benchmark because it showed the clearest difference between O3 and O3_no_vec in the runtime and energy results Under O3, gemm had one of the fastest runtimes and the lowest estimated energy value. Under O3_no_vec, runtime and estimated energy were noticeably higher. This made gemm a useful example for checking whether the generated loop code changed in a way that could help explain the measured energy difference.

```
.LBB0_22:
    add     x5, x2, x3
    add     x4, x14, x3
    ldr     q4, [x5, #18320]
    ldr     q5, [x4, #18336]
    fmla    v5.2d, v4.2d, v3.2d
    str     q5, [x4, #18336]
    ldr     q4, [x5, #18336]
    ldr     q5, [x4, #18352]
    fmla    v5.2d, v4.2d, v3.2d
    str     q5, [x4, #18352]
    ldr     q4, [x5, #18352]
    ldr     q5, [x5, #18368]
    ldr     q6, [x4, #18368]
    ldr     q7, [x4, #18384]
    fmla    v6.2d, v4.2d, v3.2d
    fmla    v7.2d, v5.2d, v3.2d
    str     q6, [x4, #18368]
    str     q7, [x4, #18384]
    add     x3, x3, #64
    b       .LBB0_22
```

LISTING 15: SELECTED GEMM O3 INNER-LOOP ASSEMBLY EXCERPT

The O3 excerpt shows a compact loop body. The ldr instructions load values from memory, the fmla instructions perform fused multiply-add operations, and the str instructions store the updated values back to memory. The important point is that these operations are grouped closely together inside the same loop body before the branch returns to the start of the loop.

The use of q registers and fmla v*.2d instructions shows that the O3 loop is using vector operations. This means that one instruction can operate on more than one double-precision value at a time. This is different from a simple scalar loop, where each instruction usually works on one value at a time.

```
.LBB2_10:
    ldr     d0, [sp, #56]
    ldr     x8, [sp, #32]
    ldrsw   x9, [sp, #20]
    mov     x10, #20800
    mul     x9, x9, x10
    add     x8, x8, x9
    ldrsw   x9, [sp, #12]
    ldr     d1, [x8, x9, lsl #3]
    fmul    d0, d0, d1

    ldr     x8, [sp, #24]
    ldrsw   x9, [sp, #12]
    mov     x10, #18400
    mul     x9, x9, x10
    add     x8, x8, x9
    ldrsw   x9, [sp, #16]
    ldr     d1, [x8, x9, lsl #3]

    ldr     x8, [sp, #40]
    ldrsw   x9, [sp, #20]
    mul     x9, x9, x10
    add     x8, x8, x9
    ldrsw   x9, [sp, #16]
    ldr     d2, [x8, x9, lsl #3]
    fmadd   d0, d0, d1, d2
    str     d0, [x8, x9, lsl #3]

    ldr     w8, [sp, #16]
    add     w8, w8, #1
    str     w8, [sp, #16]
    b       .LBB2_9
```

LISTING 16: SELECTED GEMM O0 INNER-LOOP ASSEMBLY EXCERPT

The O0 excerpt shows a more direct scalar loop structure. It uses scalar d registers rather than vector q registers. It also contains repeated address calculations, stack accesses, and loop-control operations. This suggests that the O0 version follows the original source-level loop more closely and does not apply the same level of loop restructuring.

Compared with O0, the O3 loop performs more useful work before each branch back to the loop header. It also uses vector fused multiply-add instructions, which are better suited to repeated matrix multiplication work. This supports the measured result for gemm, where O3 produced much lower runtime and estimated energy than O0 and O3_no_vec.

This evidence should still be interpreted carefully. The assembly comparison shows loop-level optimisation and scheduling behaviour that is consistent with software-pipelining-related effects. However, it does not prove that software pipelining alone caused the energy reduction, because O3 can also include vectorisation, instruction scheduling, register allocation, and loop restructuring. The safer conclusion is that LLVM changed the loop execution structure in ways that improved energy efficiency, with software pipelining being one possible contributor.

14.4 Summary of Findings

The results show that LLVM optimisation improved estimated energy efficiency across the tested PolyBench kernels. Every optimised configuration reduced estimated energy compared with the O0 baseline, with an average reduction of approximately 54.8%. The largest improvement was seen in gemm, where O3 reduced estimated energy by approximately 82.8%, while the smallest improvement was seen in 3mm, where O2 and O3 reduced energy by approximately 29.6%. This shows that optimisation was beneficial overall, but the scale of improvement depended on the workload.

The runtime and power results explain this pattern. O0 generally drew less average power than the optimised variants, but it ran for much longer. The optimised configurations often increased average power draw, but the reduction in runtime was large enough to reduce total estimated energy. This confirms that energy efficiency could not be judged from power alone, and that runtime was the main driver of the energy reductions in this experiment.

The O3 and O3_no_vec comparison showed that aggressive optimisation behaved differently across kernels. O3 was more energy efficient for gemm, corr, and 2mm, while O3_no_vec was slightly more efficient for 3mm. The strongest difference was in gemm, where disabling vectorisation increased estimated energy by 73.07% compared with O3. This suggests that regular matrix multiplication benefited strongly from vectorised or aggressive loop-level code generation, while more complex workloads did not respond in the same way.

The assembly inspection supported the measured results. In gemm, the O3 assembly used vector registers, fused multiply-add instructions, and fewer branch/control instructions than O0. This shows that LLVM changed the loop body into a more efficient form, which is consistent with the large runtime and energy reduction measured for that benchmark.

Overall, the findings support the conclusion that LLVM loop-level optimisation improved estimated energy efficiency on the Raspberry Pi 5. However, the evidence does not prove that software pipelining alone caused the improvement. The assembly results show behaviour consistent with loop scheduling and software-pipelining-related effects, but O3 also includes vectorisation, instruction scheduling, register allocation, and loop restructuring. Therefore, software pipelining should be treated as one possible contributor within the broader set of LLVM optimisation effects.

15 Conclusion And Critical Evaluation

15.1 Further Work

Further work could improve the project by addressing the main limitations of the experimental method. The first improvement would be to use external power measurement hardware. This project used Raspberry Pi 5 PMIC readings, which were useful for system-level comparison, but they did not isolate CPU-only energy consumption. An external power meter or dedicated measurement board would allow the PMIC readings to be validated and would provide more accurate energy measurements.

A second improvement would be to isolate individual LLVM optimisation passes more directly. This project compared O0, O2, O3, and O3_no_vec, which was useful for comparing practical compiler configurations. However, these optimisation levels apply several transformations at the same time. Future work could use LLVM pass control, optimisation remarks, or pass debugging output to enable and disable specific loop optimisation passes. This would make it easier to determine whether software pipelining, vectorisation, loop unrolling, or instruction scheduling was responsible for a particular energy change.

The benchmark set could also be expanded. The selected PolyBench kernels were suitable because they are loop-intensive, but they do not represent every type of workload. Future work could include more PolyBench kernels and real embedded applications to test whether the findings apply beyond the selected benchmarks.

The analysis could also be strengthened with more detailed statistics. This project used repeated runs, means, standard deviations, and percentage comparisons. Future work could add confidence intervals, outlier detection, and statistical significance testing. This would make it easier to decide whether small differences between optimisation variants are meaningful or caused by measurement noise.

A final improvement would be to extend the assembly-level investigation. This project used assembly line counts, instruction indicators, and selected loop excerpts to support the energy findings. Future work could combine this with LLVM optimisation reports, hardware performance counters, and detailed dependency analysis. This would provide stronger evidence about which compiler transformations affected runtime and energy consumption.

15.2 Reflection

This project improved my understanding of compiler optimisation, energy measurement, and experimental design. At the start of the project, the focus was mainly on software pipelining as a specific optimisation technique. During the project, it became clear that isolating software pipelining from the wider LLVM optimisation pipeline was difficult because standard optimisation levels apply several transformations together. This changed the direction of the work from trying to prove one optimisation in isolation to making a more careful argument about LLVM loop-level optimisation.

One important lesson was that performance and energy efficiency are related, but not identical. The optimised configurations often drew more average power than O0 while running, but still used less estimated energy because they completed the workloads faster. This showed that average power alone is not enough to judge energy efficiency.

The project also showed the importance of repeatable experimental design. The build, controller, and logging scripts helped keep the experiment consistent by automating compilation, execution, cooldown periods, state tracking, and CSV logging. This reduced manual work and made the results easier to analyse. However, the project also showed that real hardware measurement is difficult to fully control because background system activity, temperature, and power-management behaviour can still affect results.

The original proposal was improved during the project. Moving from a custom loop benchmark to selected PolyBench kernels made the work stronger because PolyBench provided established loop-intensive benchmarks. The final project was therefore broader and more evidence-based than the original idea. It did not prove software pipelining as a single cause, but it did show that LLVM optimisation changed generated code and improved estimated energy efficiency for the tested workloads.

Overall, the project developed practical skills in scripting, measurement, data processing, graphing, and assembly-level interpretation. It also improved my ability to evaluate results critically rather than only reporting whether one configuration was faster or slower.

15.3 Final Conclusion

This project achieved its main aim by developing a working experimental framework for measuring the runtime, power, and estimated energy effects of selected LLVM optimisation configurations on real low-power hardware. The framework successfully compiled PolyBench kernels under different optimisation settings, executed them on a Raspberry Pi 5, collected PMIC-based power readings, and produced structured CSV data for analysis.

The results showed that LLVM optimisation improved estimated energy efficiency across the tested loop-intensive workloads. Although the optimised configurations generally increased average power draw compared with O0, they reduced runtime by a much larger margin. As a result, total estimated energy consumption decreased for every tested benchmark. This shows that energy efficiency cannot be judged from power consumption alone, as execution time had a major influence on the final energy results.

The assembly-level inspection supported the measured findings. In the gemm example, the O3 assembly used vector registers, fused multiply-add instructions, and a more compact loop body than the O0 version. This matched the experimental results, as gemm showed the strongest reduction in both runtime and estimated energy under optimisation. However, the assembly evidence did not prove that software pipelining alone caused these improvements. Since O3 applies several transformations at the same time, including vectorisation, instruction scheduling, register allocation, and loop restructuring, software pipelining should be treated as one possible contributor within a wider set of LLVM loop-level optimisation effects.

The project also had clear limitations. The Raspberry Pi 5 PMIC readings provided practical system-level power measurements, but they did not isolate CPU-only energy consumption. The benchmark set was appropriate for loop-intensive analysis, but it was limited to four selected PolyBench kernels. In addition, energy was estimated using average power multiplied by runtime, which provided a consistent comparison method but was less precise than sample-by-sample energy integration.

Overall, the findings show that LLVM loop-level optimisation can improve estimated energy efficiency for the tested workloads on the Raspberry Pi 5, mainly by reducing execution time enough to outweigh increases in average power draw. However, the exact contribution of software pipelining could not be isolated without deeper compiler-pass analysis. The safest conclusion is therefore that LLVM optimisation changed generated code and energy behaviour in measurable ways, while software pipelining remains a plausible but not individually proven contributor.

16 Appendix

16.1 Originality & Use of Generative AI Statement

I understand that to use the work and ideas of others, including AI-generated output, without full acknowledgement, is academic misconduct.

I confirm that this coursework submission is all my own, original work and that all sources, summaries, paraphrases and quotes are fully referenced as required by the LSBU Academic Regulations.

DECLARATION OF AI USE:
I **DID** use Generative AI technology in the development, writing, or editing of this assignment. (delete as appropriate)

If you did use Generative AI, please provide detailed responses to the following items:

1. Tool used was: ChatGPT

2. Purpose:

   - Editing:I used ChatGPT to improve sentence structure, clarity, and wording. It helped me reduce repeated phrasing and make some sections easier to read.
   - Grammar checking: I used ChatGPT to identify and correct grammar, punctuation, and spelling errors throughout the report.
   - Caption and formatting: I used ChatGPT to improve the wording of figure captions, table captions, and code listing captions so they were more consistent across the report.
   - Feedback: I used ChatGPT to get feedback on the structure and clarity of specific sections, including the analysis, summary of findings, conclusion, abstract, acknowledgements, and AI statement.
   - I used ChatGPT to create a high-level project diagram based on my own project workflow. The diagram represented the build script, controller script, logging script, benchmark binaries, PMIC readings, CSV logs, and analysis stage.

3. Provide 2-3 examples of prompts you used when using AI tools for this assignment.

"Can you check whether there are inconsistencies between my build script, controller script, and logging script?"

"Can you help me write a caption for the assembly line count graph?"

"Can you review my bibliography and tell me if there are duplicates or Harvard referencing inconsistencies?"

"Can you check the structure of my sentence?"

4. Reflection:

Generative AI helped improve the clarity, structure, and presentation of my report. I used it to check grammar, reduce repeated wording, improve sentence flow, and make captions more consistent. It was also useful for reviewing sections such as the analysis, findings, conclusion, and appendix, where clear explanation was important.

However, I understand that Generative AI can produce wording that sounds correct but does not always reflect the actual experiment, code, or results. For this reason, I checked the suggestions against my own implementation, data, and project evidence before using them. I did not use AI to generate my experimental results or make conclusions independently. The final interpretation and submitted work remain my own

By including this statement in my coursework submission, I attest that the information provided in this Originality and use of Generative AI Statement is accurate and complete to the best of my knowledge. I understand that providing false information is a violation of the LSBU Academic Regulations and may result in academic and/or disciplinary consequences.

Ethical Considerations

London South Bank University

EST 1892

School of Engineering

BSc FINAL YEAR PROJECT
ETHICAL CONSIDERATIONS
2025-26

YOUR DETAILS

Name of student: Abdihakim Burale

Supervisor: Oswaldo Cadenas

Project title: LLVM Optimisation for energy efficiency via software pipelining case study

Main aim of project:

To investigate whether LLVM compiler optimisations can improve software energy efficiency on low-power hardware, with a particular focus on software-pipelining related loop optimisations.

CONTACT WITH OTHERS

Will your project bring you into contact with other people (e.g. via an online survey)?

Yes ☐ No ☒

If you answered "No", sign the section below and submit this page only to your supervisor for countersigning, otherwise complete the whole form prior to submission. Also, if you answered "No" then only this page needs to be included as an appendix to your dissertation.

YOUR SIGNATURE

Signature: Abdihakim Burale Date: 05/05/2026

ETHICAL APROVAL

(To be completed by your supervisor)

I have checked the above for accuracy and I am satisfied that the information provided is an accurate reflection of the intended study.

There are no ethical issues causing my concern ☐

Signature: Date: 06-05-2026

Name (please print): OSWALDO CADENAS

16.2 Full Code Listings

16.2.1Build_binaries.Sh

```
#!/usr/bin/env bash
set -e

BASE="$HOME/dev/power_project"
POLY="$BASE/PolyBenchC-4.2.1"
OUT="$BASE/binaries"
LOG="$BASE/binaries/build_manifest.txt"

mkdir -p "$OUT/gemm" "$OUT/2mm" "$OUT/3mm" "$OUT/corr"
rm -f "$LOG"

echo "Building PolyBench LLVM binaries (EXTRALARGE_DATASET)..." | tee -a
"$LOG"

UTIL="$POLY/utilities/polybench.c"
COMMON="-mcpu=cortex-a72 -DPOLYBENCH_TIME -DEXTRALARGE_DATASET -I
$POLY/utilities"

compile () {
    SRC=$1
    NAME=$2
    DIR=$3

    echo "Compiling $NAME..."

    echo "=== $NAME ===" >> "$LOG"

    clang -O0 $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O0"
    echo "O0: ${NAME}_O0" >> "$LOG"

    clang -O2 $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O2"
    echo "O2: ${NAME}_O2" >> "$LOG"

    clang -O3 $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O3"
    echo "O3: ${NAME}_O3" >> "$LOG"

    clang -O3 -fno-vectorize -fno-slp-vectorize $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O3_no_vec"
    echo "O3_no_vec: ${NAME}_O3_no_vec" >> "$LOG"
```

```
    clang -O3 -funroll-loops $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O3_unroll"
    echo "O3_unroll: ${NAME}_O3_unroll" >> "$LOG"

    clang -O3 -fno-unroll-loops $COMMON "$UTIL" "$SRC" -lm \
        -o "$OUT/$DIR/${NAME}_O3_no_unroll"
    echo "O3_no_unroll: ${NAME}_O3_no_unroll" >> "$LOG"
}

compile "$POLY/linear-algebra/blas/gemm/gemm.c" "gemm" "gemm"
compile "$POLY/linear-algebra/kernels/2mm/2mm.c" "2mm" "2mm"
compile "$POLY/linear-algebra/kernels/3mm/3mm.c" "3mm" "3mm"
compile "$POLY/datamining/correlation/correlation.c" "corr" "corr"

echo "Build complete."
```

16.2.2 run_experiments.py

```
#!/usr/bin/env python3

import subprocess
import time
import os
import json

BASE_DIR = "/home/hakeem/dev/power_project"
LOGGER = os.path.join(BASE_DIR, "scripts/log_power_final.py")

KERNELS = ["gemm", "2mm", "3mm", "corr"]
VARIANTS = ["O0", "O2", "O3", "O3_no_vec"]

REPEATS = 4
DURATION = 300
COOLDOWN = 120

STATE_FILE = os.path.join(BASE_DIR, "experiment_state.json")
STOP_FILE = os.path.join(BASE_DIR, "STOP")


# ---------- LOAD STATE ----------
if os.path.exists(STATE_FILE):
    with open(STATE_FILE, "r") as f:
        completed = set(tuple(x) for x in json.load(f))
else:
    completed = set()
```

```python
def save():
    with open(STATE_FILE, "w") as f:
        json.dump(list(completed), f)


def upload_logs():
    print("\n[CTRL][UPLOAD] Syncing logs to Google Drive...")
    subprocess.run([
        "rclone",
        "copy",
        os.path.join(BASE_DIR, "logs"),
        "gdrive:logs"
    ])
    print("[CTRL][UPLOAD COMPLETE]")


# ---------- MAIN LOOP ----------
for kernel in KERNELS:
    for variant in VARIANTS:
        for run in range(REPEATS):

            if os.path.exists(STOP_FILE):
                print("[CTRL][STOP] Detected STOP file. Exiting cleanly.")
                exit(0)

            key = (kernel, variant, run)

            if key in completed:
                print("[CTRL][SKIP]", key)
                continue

            print(f"\n[CTRL][RUN START] {key}\n")

            subprocess.run([
                "python3",
                LOGGER,
                kernel,
                variant,
                str(DURATION)
            ], check=True)

            completed.add(key)
            save()

            print(f"[CTRL][COOLDOWN] {COOLDOWN}s\n")
            time.sleep(COOLDOWN)

        # upload after finishing ALL repeats of a variant
        upload_logs()

print("\n[CTRL] ALL EXPERIMENTS COMPLETE")
```

16.2.3 Log_power_final.py

```
#!/usr/bin/env python3

import subprocess
import time
import os
import csv
import sys
import datetime
import uuid

BASE_DIR = "/home/hakeem/dev/power_project"
BINARY_DIR = os.path.join(BASE_DIR, "binaries")
LOG_DIR = os.path.join(BASE_DIR, "logs")

INTERVAL = 0.5
PRERUN_TIME = 10

IMPORTANT_RAILS = ["VDD_CORE", "3V3_SYS", "1V8_SYS", "1V1_SYS"]


def read_cpu():
    try:
        with open("/sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq") as f:
            freq = int(f.read().strip())
    except:
        freq = -1
    return freq


def read_temp():
    try:
        out = subprocess.run(
            ["vcgencmd", "measure_temp"],
            stdout=subprocess.PIPE,
            text=True
        )
        return float(out.stdout.split("=")[1].replace("'C", ""))
    except:
        return -1.0


def read_throttle():
    try:
        out = subprocess.run(
            ["vcgencmd", "get_throttled"],
            stdout=subprocess.PIPE,
```

```
            text=True
        )
        return int(out.stdout.split("=")[1], 16)
    except:
        return -1


def read_fan():
    try:
        with open("/sys/class/hwmon/hwmon0/pwm1") as f:
            return int(f.read().strip())
    except:
        return -1


def read_pmic():
    try:
        result = subprocess.run(
            ["vcgencmd", "pmic_read_adc"],
            stdout=subprocess.PIPE,
            text=True
        )

        lines = result.stdout.splitlines()

        currents = {}
        voltages = {}

        for line in lines:
            if "=" not in line:
                continue

            parts = line.split()
            label = parts[0]
            raw = parts[1].split("=")[1]

            if raw.endswith("A"):
                currents[label[:-2]] = float(raw[:-1])
            elif raw.endswith("V"):
                voltages[label[:-2]] = float(raw[:-1])

        row = {}
        total = 0.0

        for rail in IMPORTANT_RAILS:
            v = voltages.get(rail, 0.0)
            i = currents.get(rail, 0.0)
            p = v * i

            row[f"{rail}_V"] = v
            row[f"{rail}_A"] = i
```

```
            row[f"{rail}_W"] = p

            total += p

        return row, total

    except:
        return {f"{r}_{x}": 0 for r in IMPORTANT_RAILS for x in ["V", "A",
"W"]}, 0.0


def run_experiment(kernel, variant, duration):

    binary_path = os.path.join(BINARY_DIR, kernel, f"{kernel}_{variant}")

    if not os.path.exists(binary_path):
        print(f"[ERROR] Missing binary: {binary_path}")
        sys.exit(1)

    os.makedirs(LOG_DIR, exist_ok=True)

    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    csv_path = os.path.join(LOG_DIR, f"{kernel}_{variant}_{timestamp}.csv")

    print("\n[LOGGER START]")
    print(binary_path)
    print(f"[PRERUN] warming for {PRERUN_TIME}s")

    fieldnames = [
        "kernel",
        "variant",
        "run_id",
        "run_index",
        "time",
        "run_time",
        "total_power",
        "cpu_freq"
    ]

    for r in IMPORTANT_RAILS:
        fieldnames += [f"{r}_V", f"{r}_A", f"{r}_W"]

    # NEW metrics
    fieldnames += ["cpu_temp", "throttled", "fan_pwm"]

    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()

        start_global = time.time()
```

```
        run_index = 0
        run_id = str(uuid.uuid4())

        proc = subprocess.Popen([binary_path])
        run_start = time.time()

        time.sleep(PRERUN_TIME)

        while True:

            now = time.time()
            t = now - start_global

            if t > duration:
                break

            # restart when PolyBench finishes
            if proc.poll() is not None:
                print(f"[RUN COMPLETE] exec_run={run_index}
duration={time.time() - run_start:.2f}s")

                proc.wait()
                run_index += 1
                run_id = str(uuid.uuid4())

                print(f"[RESTART] exec_run={run_index}")

                proc = subprocess.Popen([binary_path])
                run_start = time.time()

            run_time = now - run_start

            freq = read_cpu()
            temp = read_temp()
            throttle = read_throttle()
            fan = read_fan()

            pmic, total = read_pmic()

            row = {
                "kernel": kernel,
                "variant": variant,
                "run_id": run_id,
                "run_index": run_index,
                "time": round(t, 3),
                "run_time": round(run_time, 3),
                "total_power": total,
                "cpu_freq": freq,
                "cpu_temp": temp,
                "throttled": throttle,
                "fan_pwm": fan
```

```
            }

            row.update(pmic)

            writer.writerow(row)

            print(
                f"{kernel} {variant} | exec_run={run_index} | "
                f"t={t:.1f}s | run_t={run_time:.2f}s | "
                f"P={total:.2f}W | T={temp:.1f}C"
            )

            time.sleep(INTERVAL)

        proc.terminate()
        proc.wait()

    print(f"Saved: {csv_path}")


if __name__ == "__main__":
    kernel = sys.argv[1]
    variant = sys.argv[2]
    duration = int(sys.argv[3])
    run_experiment(kernel, variant, duration)
```

16.3 Bibliography

Ahmed, K.M.U., Bollen, M.H.J. and Alvarez, M. (2021). A review of data centers energy consumption and reliability modeling. *IEEE Access*, 9, pp.152536–152563. doi:10.1109/ACCESS.2021.3125092.

Allan, V.H., Jones, R.B., Lee, R.M. and Allan, S.J. (1995). Software pipelining. *ACM Computing Surveys*, 27(3), pp.367–432. doi:10.1145/212094.212131.

Carr, S., Ding, C. and Sweany, P. (1996). Improving software pipelining with unroll-and-jam. In: *Proceedings of HICSS-29: 29th Hawaii International Conference on System Sciences*. IEEE, pp.183–192. doi:10.1109/HICSS.1996.495462.

Daud, S., Ahmad, R.B. and Murthy, N.S. (2008). The effects of compiler optimizations on embedded system power consumption. In: *Proceedings of the International Conference on Electronic Design*. Penang, Malaysia, 1–3 December. IEEE.

D'Silva, V., Payer, M. and Song, D. (2015). The correctness-security gap in compiler optimization. In: *Proceedings of the IEEE Symposium on Security and Privacy Workshops (SPW 2015)*. IEEE. doi:10.1109/SPW.2015.33.

Esakkimuthu, G., Vijaykrishnan, N., Kandemir, M. and Irwin, M.J. (2000). Memory system energy: Influence of hardware-software optimizations. In: *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED 2000)*. IEEE, pp.244–249. doi:10.1109/LPE.2000.876795.

Georgiou, K., Blackmore, C., Xavier-de-Souza, S. and Eder, K. (2018). Less is more: Exploiting the standard compiler optimization levels for better performance and energy consumption. In: *Proceedings of the 21st International Workshop on Software and Compilers for Embedded Systems*. ACM. doi:10.1145/3207719.3207727.

Kandemir, M., Vijaykrishnan, N., Irwin, M.J. and Ye, W. (2000). Influence of compiler optimizations on system power. In: *Proceedings of the 27th International Symposium on Computer Architecture (ISCA 2000)*. Vancouver, Canada: ACM, pp.304–313. doi:10.1145/337292.337425.

Laganà, D., Mastroianni, C., Meo, M. and Renga, D. (2018). Reducing the operational cost of cloud data centers through renewable energy. *Algorithms*, 11(10), p.145. doi:10.3390/a11100145.

Lattner, C. (2002). *LLVM: An infrastructure for multi-stage optimization*. Master's thesis. University of Illinois at Urbana-Champaign. Available at: https://llvm.org/pubs/2002-12-LattnerMSThesis.html (Accessed: 11 December 2025).

Mahdavinejad, M.S., Rezvan, M., Barekatain, M., Adibi, P., Barnaghi, P. and Sheth, A.P. (2019). Machine learning for internet of things data analysis: A survey. *Digital Communications and Networks*, 4(3), pp.161–175. doi:10.1016/j.dcan.2017.10.002.

Maleki, S., Gao, Y., Garzar, M.J., Wong, T. and Padua, D. (2011). An evaluation of vectorizing compilers. In: *Proceedings of the 20th International Conference on Parallel Architectures and Compilation Techniques (PACT 2011)*. IEEE. doi:10.1109/PACT.2011.68.

Pan, H., Zhang, H., Xing, M. and Wu, Y. (2025). A hybrid, knowledge-guided evolutionary framework for personalized compiler auto-tuning. *arXiv preprint arXiv:2510.14292*. Available at: https://arxiv.org/abs/2510.14292 (Accessed: 11 December 2025).

Pugh, W. (1991). Uniform techniques for loop optimization. In: *Proceedings of the International Conference on Supercomputing*. Cologne, Germany: ACM, pp.341–352. doi:10.1145/109025.109108.

Ross, A. and Christie, L. (2024). *Energy consumption of ICT*. London: Parliamentary Office of Science and Technology. Available at: https://post.parliament.uk/research-briefings/post-pn-0677/ (Accessed: 9 December 2025).

Shehabi, A., Smith, S., Hubbard, A., Newkirk, A., Lei, N., Abu, M., Holecek, B., Koomey, J., Masanet, E. and Sartor, D. (2024). *2024 United States data center energy usage report*. Berkeley: Lawrence Berkeley National Laboratory. Available at: https://escholarship.org/uc/item/32d6m0d1 (Accessed: 9 December 2025).

Taufer, M. and Rosenberg, A.L. (2016). Scheduling DAG-based workflows on single cloud instances: High-performance and cost effectiveness with a static scheduler. *The International Journal of High Performance Computing Applications*, 31(1), pp.19–31. doi:10.1177/1094342015594518.

Wu, J., Zheng, J., Yang, Z. and Yu, Z. (2025). Compiler optimization testing based on optimization-guided equivalence transformations. *arXiv preprint arXiv:2504.04321*. Available at: https://arxiv.org/abs/2504.04321 (Accessed: 11 December 2025).

Zhao, J., Nagarakatte, S., Martin, M.M.K. and Zdancewic, S. (2012). Formalizing the LLVM intermediate representation for verified program transformations. In: *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2012)*. Philadelphia, PA: ACM, pp.427–440. doi:10.1145/2103656.2103709.